



высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Метод построения синтаксических деревьев для
ограниченного естественного русского языка на основе
наиболее употребимых грамматик

Студент ИУ7-84Б

(подпись)

И. С. Бугаков

Руководитель ВКР

(подпись)

Ю. В. Строганов

Нормоконтролер

(подпись)

(инициалы, фамилия)

2026 год



Расчетно-пояснительная записка 61 с., 16 рис., 9 таблиц, 45 источников, 4 приложения.

Ключевые слова: КОМПЬЮТЕРНАЯ ЛИНГВИСТИКА, СИНТАКСИЧЕСКИЙ АНАЛИЗ, ДЕРЕВО СОСТАВЛЯЮЩИХ, АЛГОРИТМ КОКА–ЯНГЕРА–КАСАМИ, ОГРАНИЧЕННЫЙ ЕСТЕСТВЕННЫЙ ЯЗЫК, РУССКИЙ ЯЗЫК.

Цель работы — разработка метода построения синтаксических деревьев для ограниченного естественного русского языка на основе наиболее употребимых грамматик.

В данной работе выполнен анализ предметной области автоматизированного построения синтаксических деревьев ограниченного естественного русского языка, проведен сравнительный анализ существующих методов построения синтаксических деревьев. Разработаны метод построения синтаксических деревьев и программное обеспечение реализующее описанный метод. Проведено исследование зависимости времени построения синтаксических деревьев от количества словоформ в предложении, приведена оценка точности и полноты разработанного метода.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ВВЕДЕНИЕ	8
1 Аналитический раздел	9
1.1 Анализ предметной области	9
1.2 Системы синтаксической разметки	10
1.2.1 Системы зависимостей	10
1.2.2 Системы составляющих	11
1.2.3 Системы синтаксических групп	13
1.2.4 Сравнение систем синтаксической разметки	14
1.3 Методы построения систем составляющих	15
1.3.1 Алгоритм Кока–Янгера–Касами	15
1.3.2 Алгоритм Эрли	17
1.3.3 Сравнение методов построения систем составляющих	18
1.4 Постановка задачи	19
2 Конструкторский раздел	22
2.1 Требования к разрабатываемому методу	22
2.2 Используемые структуры и форматы данных	23
2.3 Описание ключевых этапов метода	24
2.4 Диаграмма компонентов разрабатываемого ПО	31
3 Технологический раздел	32
3.1 Выбор средств реализации	32
3.2 Пользовательский интерфейс	33
3.3 Реализация ключевых этапов метода	34
3.4 Тестирование разработанного метода	40
4 Исследовательский раздел	46
4.1 Характеристики устройства и постановка исследования	46

ЗАКЛЮЧЕНИЕ	52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53
ПРИЛОЖЕНИЕ А	57
ПРИЛОЖЕНИЕ Б	60
ПРИЛОЖЕНИЕ В	61

Автоматический синтаксический анализ предложений является одной из важных задач компьютерной лингвистики [1]. Данный вид анализа позволяет решать задачи генерации текстов, автоматического реферирования, установления авторства текстов, а также исследовать синтаксические конструкции языков [2, 3].

В отечественной корпусной и прикладной лингвистике преобладающим формализмом в области синтаксического анализа являются системы зависимостей (грамматики зависимостей, деревья зависимостей, деревья подчинения), распространения которого способствовала модель «Смысл ↔ Текст» И. А. Мельчука [4]. В частности, разметку данными системами получил синтаксический подкорпус НКРЯ [5].

Существуют, однако, альтернативные формализмы, такие как системы составляющих и системы синтаксических групп [6]. Появление в недавнем времени работ, посвященных созданию корпусов с разметкой систем составляющих [7] и систем синтаксических групп [8], а также разработке методов преобразования систем зависимостей в системы составляющих [1], свидетельствует о необходимости автоматизации построения данных видов синтаксических систем, как на основе исходных текстовых данных, так и уже существующих систем зависимостей.

Цель работы — разработка метода построения синтаксических деревьев для ограниченного естественного русского языка на основе наиболее употребимых грамматик.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- проанализировать предметную область автоматизированного построения синтаксических деревьев ограниченного естественного русского языка;
- провести сравнительный анализ существующих методов построения синтаксических деревьев и сформулировать требования к разрабатываемому методу;
- формализовать постановку задачи в виде IDEF0-диаграммы;
- сформулировать ограничения предметной области;
- разработать метод построения синтаксических деревьев ограниченного естественного русского языка;
- обосновать выбор программных средств реализации и разработать программное обеспечение реализующее описанный метод;
- определить зависимость времени построения синтаксических деревьев от количества словоформ в предложении;
- оценить точность и полноту разработанного метода.

1 Аналитический раздел

1.1 Анализ предметной области

Ограниченный естественный язык – это подмножество естественного языка, текст на котором, без каких-либо усилий, воспринимается носителем исходного естественного языка, а также не требует длительного изучения для приобретения навыков составления текстов на этом языке, однако обладает сокращенным набором лексики и грамматики. Использование ограниченного естественного языка позволяет снизить время обработки естественно-языковых конструкций, без снижения уровня функциональности [9].

Разметка текста заключается в приписывании словам или предложениям меток (тэгов) в соответствии с характером разметки: морфологические, синтаксические, семантические и другие [10].

Джеффри Лич приводит следующие принципы, которым должна подчиняться разметка общедоступного корпуса [11]:

- разметка должна основываться на доступной для пользователя в виде руководства или инструкции схеме анализа, в которой должно быть мотивированно введение каждого параметра;
- разметка общедоступного корпуса должна опираться на общеизвестную систему правил и классификаций;
- должно быть ясно кто и как разрабатывает схему аннотации и каковы ограничения, например юридические или технические, при использовании корпусом [12].

Синтаксическая разметка является результатом разбора, выполняемого на основе данных морфологического анализа. Данный вид разметки описывает синтаксические связи и конструкции, образованные лексическими единицами [10, 12]. Тексты, получившие синтаксическую разметку известны в англоязычной литературе как treebanks [13]. Пример визуализации синтаксических зависимостей, отображаемых синтаксической разметкой, представлен на рисунке 1.1.

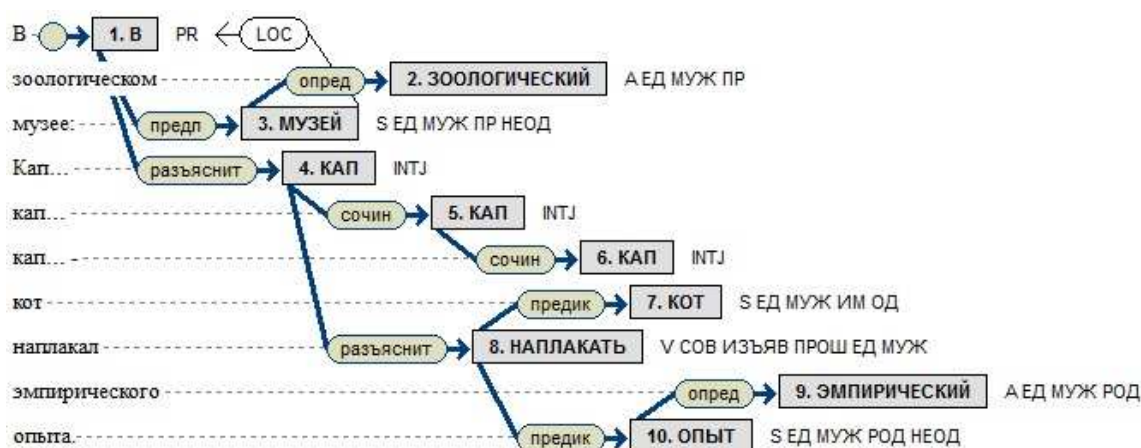


Рисунок 1.1 — Визуализация синтаксических зависимостей в НКРЯ

В корпусной лингвистике наблюдается большое разнообразие синтаксических теорий и формализмов, среди которых наиболее распространенными являются следующие [6, 10]:

- системы зависимостей;
- системы непосредственных составляющих;
- системы синтаксических групп.

1.2 Системы синтаксической разметки

1.2.1 Системы зависимостей

Если Π — конечное линейное упорядоченное множество, то всякое дерево D , для которого Π служит множеством узлов, называется деревом синтаксического подчинения (зависимостей) на Π . Если Π — множество точек некоторой цепочки x , говорят, что D — дерево синтаксического подчинения (зависимостей) для x [6]. Размеченным деревом подчинения (зависимостей) называется упорядоченная четверка 1.1:

$$\langle \Pi, \rightarrow, Z, \psi \rangle, \quad (1.1)$$

где $\langle \Pi, \rightarrow \rangle$ — дерево зависимостей на Π ,

Z — конечное множество меток,

ψ — разметка, отображение множества дуг дерева в Z .

Таким образом дерево зависимостей представляет собой дерево, в котором вершины соответствуют отдельным словам предложения, а дуги между ними — синтаксическим подчинительным связям между словами [14]. Пример дерева зависимостей для шведского языка приведен на рисунке 1.2.

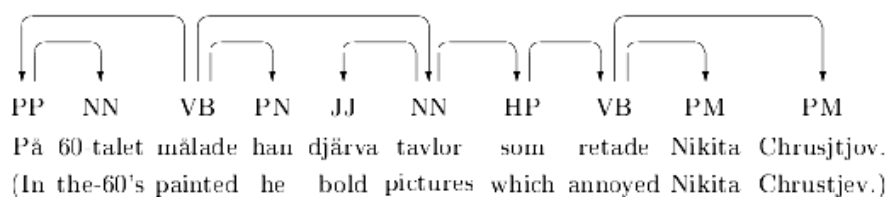


Рисунок 1.2 — Дерево зависимостей предложения на шведском языке

Дерево подчинения $\langle \Pi, \rightarrow \rangle$ называется проективным, если для любых трех его узлов α, β, γ из того, что $\alpha \rightarrow \beta$ и γ лежит между α и β , следует, что γ зависит от α . Проективные предложения соответствуют «естественному» порядку слов, тогда как непроективные более характерны художественной литературе [6]. Например, следующий оборот не является проективным: «...рассмотрев выдвинутые здесь предложения товарищем Лопаткиным...».

Требование проективности исходного предложения и, как следствие, получаемого дерева

зависимостей, является необходимым для большинства алгоритмов построения деревьев зависимостей. Алгоритм Нивре [15], являющийся основой MaltParser [16], алгоритм, используемый в LParus [17] также основываются на предположении, что предложение проективно.

Среди преимуществ систем зависимостей выделяют [6, 15, 18]:

- независимость от порядка слов в предложении;
- однозначность отношения «главное–зависимое» между словоформами.

Помимо требования проективности к недостаткам систем зависимостей относят [6, 7, 17, 18]:

- невозможность отобразить связи и иерархию словосочетаний;
- невозможность в полной мере описать неподчинительные связи;
- неоднозначность определения управляющей словоформы в некоторых случаях;
- невозможность различить адъюнкты и аргументы при предикате.

1.2.2 Системы составляющих

Множество C отрезков конечного линейно упорядоченного множества Π называется системой составляющих на Π , если оно удовлетворяет следующим условиям [6]:

- 1) C содержит в качестве элементов само Π и все одноточечные отрезки Π ;
- 2) любые два отрезка из C либо не пересекаются, либо один из них содержится в другом.

Если Π — множество точек некоторой цепочки x , говорят о системе составляющих цепочки x [6]. Пример неразмеченного дерева составляющих представлен на рисунке 1.3.

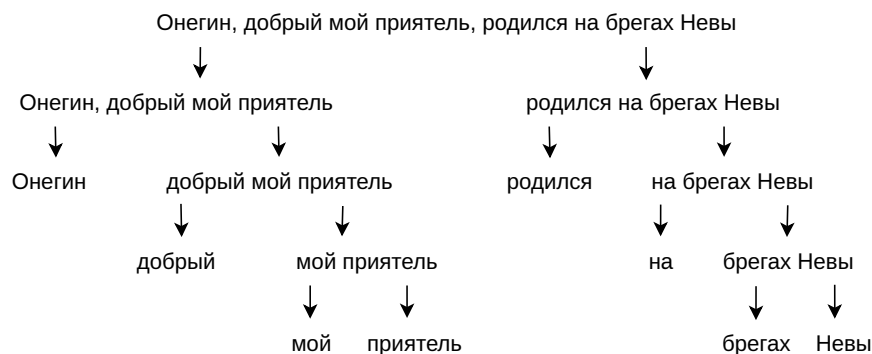


Рисунок 1.3 — Неразмеченное дерево составляющих для предложения на русском языке

Размеченной системой составляющих называют упорядоченную тройку:

$$\langle C, W, \varphi \rangle, \quad (1.2)$$

где C — система составляющих,

W — конечное множество меток,

φ — разметка, отображение C в множество всех непустых подмножеств W .

Таким образом дерево составляющих представляет собой дерево, в котором в листьях находятся слова предложения, поддеревья соответствуют входящим в предложение синтаксическим конструкциям, а дуги выражают отношение вложенности конструкций [14]. Пример размеченного дерева составляющих представлен на рисунке 1.4.

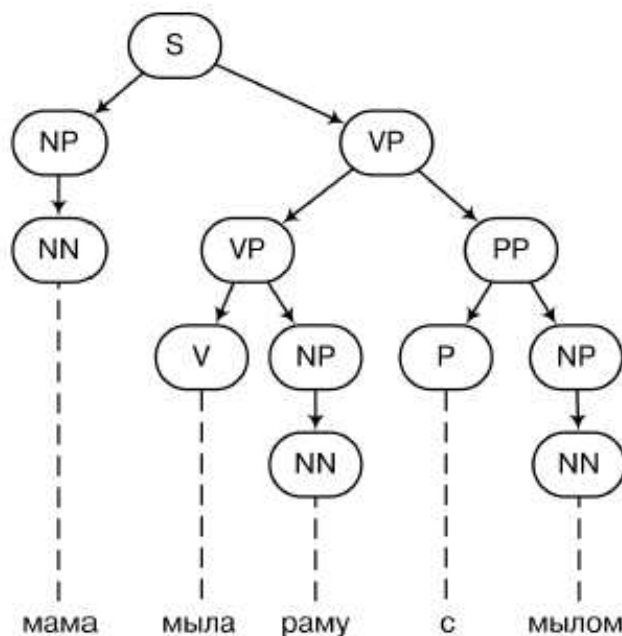


Рисунок 1.4 — Размеченное дерево составляющих для предложения на русском языке

К преимуществам системы составляющих относят [1, 7, 18, 19]:

- явную разметку фразовых категорий и синтаксических единиц;
- учет грамматически правильного порядка слов;
- построение грамматики для работы алгоритма на основе типов составляющих, что является более общим подходом, чем части речи;
- возможность различить адьюнкты и аргументы при предикате;
- возможность описать правила сочинения составляющих;
- применимость построенных деревьев для дальнейшего семантического анализа;
- возможность использования полученных синтаксических единиц и групп для прикладных задач.

Основными недостатками систем составляющих являются [1, 14, 18, 19]:

- возможность построения нескольких деревьев составляющих для одного предложения;
- увеличение количества правил в грамматике с ослаблением требований к порядку слов;
- необходимость ввода дополнительных правил и «синтетических» составляющих для учета небинарных правил и непроективности;
- высокая трудоемкость ручной разметки;
- необходимость ручного определения отношения «главное–зависимое».

Система синтаксических групп разработана и описана А. В. Гладким в работах [6, 20], как альтернатива системам составляющих и зависимостей, объединяющая используемые в них подходы, и рассматривается в более поздних работах [18, 21]. Основными преимуществами систем СГ в этих работах выделяются:

- возможность задать одновременно и вложенность групп, и иерархию подчинения отдельных словоформ;
- возможность задать разрывные и непроективные группы без введения дополнительных конструкций и правил;
- отсутствие необходимости адаптировать данную систему к русскому языку: А. В. Гладкий изначально разрабатывал ее в приложении к русскому языку.

Недостатками системы СГ следующие [6, 18, 20, 21]:

- множество групп получается в ходе построения системы и не является счетным;
- для одного предложения возможно получить несколько альтернативных систем СГ;
- лингвистические правила для построения систем синтаксических групп должны совмещать правила построения систем зависимостей;
- ССГ в том виде, в котором ее описал А. В. Гладкий не покрывает в полной мере сочинительных конструкций;
- системы СГ не получили широкого распространения в корпусной лингвистике русского языка.

1.2.4 Сравнение систем синтаксической разметки

Для сравнения систем синтаксической разметки были выделены следующие критерии:

- возможность выделения фразовых категорий;
- поддержка непроективных предложений;
- возможность построения альтернативных деревьев разбора предложения;
- использование системы в качестве источника для порождающей грамматики;
- наличие общедоступных корпусов русского языка с разметкой данным видом систем.

Сравнение систем синтаксической разметки по приведенным критериям представлено в таблице 1.1.

Таблица 1.1 — Сравнение систем синтаксической разметки

	Системы зави- сисмостей	Системы составля- ющих	Системы синтакси- ческих групп
Выделение фразовых категорий	Нет	Да	Да
Поддержка непроек- тивных предложений	Нет	Да	Да
Возможность построе- ния нескольких дере- вьев разбора	Нет	Да	Да
Использование систе- мы как порождающей грамматики	Да	Да	Нет
Общедоступные кор- пуса с разметкой та- ким видом систем	Да	Да	Нет

Таким образом, системы составляющих являются наиболее предпочтительными для реализации, как наиболее полно отражающие синтаксические структуры русского языка и в то же время, являющиеся в достаточной степени распространенными в отечественной корпусной лингвистике [18]. В [7, 10] отмечается, что хотя общедоступные корпуса русского языка с разметкой систем составляющих распространены гораздо меньше, чем общедоступные корпуса с разметкой систем зависимостей, разметка систем составляющих, создание корпусов и средств автоматизации выполнения данного вида разметки является важным для лингвистических исследований.

1.3 Методы построения систем составляющих

1.3.1 Алгоритм Кока–Янгера–Касами

В [19] описан алгоритм Кока—Янгера—Касами (СҮК) построения дерева составляющих для контекстно-свободной грамматики. Перед разбором грамматику приводят к нормальной форме Хомского (НФХ): правила имеют вид $A \rightarrow BC$, где в правой части находятся два нетерминала, или $A \rightarrow w$, где w — терминал [19]. Любую контекстно-свободную грамматику можно преобразовать в слабо эквивалентную грамматику в НФХ; при этом структура деревьев разбора сохраняется с точностью до введения вспомогательных нетерминалов [19].

Алгоритм использует подход динамического программирования. Для предложения из n словоформ заполняют верхнетреугольную часть таблицы dp размером $(n+1) \times (n+1)$. Индексы $i, j = \overline{0, n}$ соответствуют позициям между словами (от нуля до n); ячейка $dp_{i,j}$ при $i < j$ хранит множество нетерминалов, порождающих подстроку слов с i -й по j -ю позицию. На «диагонали» длины 1, то есть в $dp_{i,i+1}$, после чтения i -го слова помещают нетерминалы (и при необходимости

частеречные категории) по лексическим правилам $A \rightarrow w_i$.

Поскольку в НФХ у каждой внутренней вершины дерева составляющих ровно два потомка, нетерминал A может появиться в $dp_{i,j}$ только при некотором разбиении $i < k < j$ и правиле $A \rightarrow BC$, где $B \in dp_{i,k}$ и $C \in dp_{k,j}$. Формально для всех $i < k < j$ выполняется выражение 1.4.

$$dp_{i,j} = dp_{i,j} \cup \{A \mid A \rightarrow BC \in P, B \in dp_{i,k}, C \in dp_{k,j}\}. \quad (1.4)$$

Пример заполнения таблицы dp представлен на рисунке 1.6.

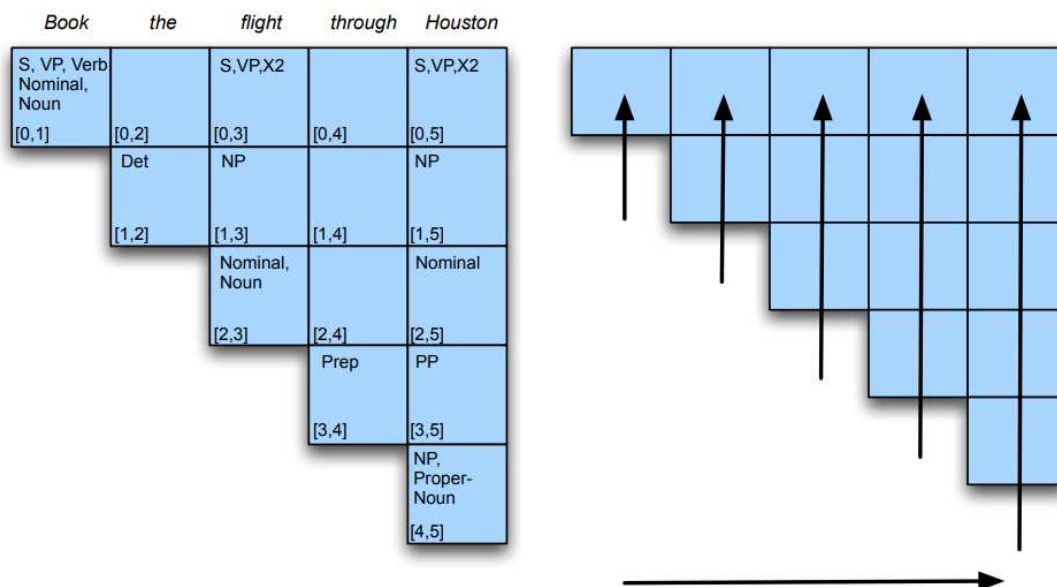


Рисунок 1.6 — Пример заполнения таблицы для предложения «Book the flight through Houston»

Заполнение таблицы dp производится слева направо, снизу-вверх, к моменту вычисления $dp_{i,j}$ известны значения всех $dp_{l,j}$, $l < i$ и $dp_{i,m}$, $m < j$, соответственно могут быть проверены все возможные разбиения интервала $[i, j]$. Варианты разбора входного предложения, таким образом, находятся в $dp_{0,n}$. Выбор верного варианта разбора среди представленных в $dp_{0,n}$ осуществляется вручную или предобученным классификатором.

Для построения дерева составляющих по заполненной таблице dp при добавлении A в $dp_{i,j}$ из правила $A \rightarrow BC$ и разбиения k сохраняют указатели на $dp_{i,k}$ и $dp_{k,j}$ и использованное правило. Восстановление дерева — рекурсивный обход от выбранного $S \in dp_{0,n}$. При нескольких разборах выбор одного дерева выносится за рамки классического СУК и может выполняться вручную, по дополнительным ограничениям грамматики или с помощью предобученной модели, оценивающей правдоподобность конститuent [19].

Вычислительная сложность распознавания для грамматики фиксированного размера составляет $O(n^3)$ при длине предложения n [22].

1.3.2 Алгоритм Эрли

Алгоритм Эрли [19, 23] относится к методам синтаксического разбора контекстно-свободных грамматик также с использованием подхода динамического программирования.

Для предложения из n словоформ данный алгоритм формирует массив dp множеств состояний размера $n + 1$. dp_i соответствует множеству возможных состояний частично построенного дерева зависимостей после обработки i словоформ. При этом в ходе работы, алгоритм совершает единственный обход предложения слева-направо, не обрабатывая повторно уже полученные составляющие.

Состояние описывается тройкой вида 1.5:

$$\langle r, p, [i, j] \rangle, \quad (1.5)$$

где r — правило контекстно-свободной грамматики вида $A \rightarrow (X_1, X_2, \dots, X_m)$;

$p \in \overline{0, m}$ — позиция отметки в правой части правила r ;

$[i, j]$, $0 \leq i \leq j \leq n$ — границы фрагмента исходного предложения, соответствующие правой части правила r , расположенной до метки p .

Начальное состояние разбора описывается тройкой 1.6 и помещается в множество состояний dp_0 до начала работы алгоритма.

$$\langle \gamma \rightarrow S, 0, [0, 0] \rangle, \quad (1.6)$$

где γ — вспомогательный нетерминал;

S — стартовый символ грамматики.

Предложение из n словоформ считается успешно разобраным в случае, если тройка 1.7 находится в множестве состояний dp_n .

$$\langle \gamma \rightarrow S, 1, [0, n] \rangle \quad (1.7)$$

Обработка префикса предложения из i словоформ заключается в обходе каждого состояния множества dp_i и применения ровно одной из трёх нижеописанных операций в зависимости от состояния, либо пропуска данного состояния в случае неприменимости ни одной из операций. Полученные в результате применения операций новые состояния добавляются в dp_i или dp_{i+1} .

Алгоритм Эрли использует следующие операции для обработки состояний в dp :

- операция предсказания (predictor);
- операция чтения терминала (scanner);
- операция завершения разбора правила (completer).

Операция предсказания применяется в случае, если в dp_j обрабатывается состояние

грамматики вида $X_{p+1} \rightarrow Y_1, \dots Y_l$ в dp_j добавляют состояния $\langle X_{p+1} \rightarrow Y_1, \dots Y_l, 0, [j, j] \rangle$. При этом операция предсказания не изменяет j , то есть длина обработанного префикса предложения не изменяется.

Операция чтения терминала применяется в случае, если в dp_j обрабатывается состояние $\langle A \rightarrow X_1, \dots X_p, X_{p+1}, \dots X_m, p, [i, j] \rangle$ и X_{p+1} является терминалом, и словоформа на позиции $j + 1$ удовлетворяет данному терминалу. В результате применения операции чтения терминала в dp_{j+1} добавляется состояние $\langle A \rightarrow X_1, \dots X_m, p + 1, [i, j + 1] \rangle$. Таким образом, операция чтения терминала сдвигает метку p в правиле r на единицу вправо, и увеличивает длину разобранного фрагмента предложения также на единицу.

Операция завершения разбора правила применяется в случае, если в dp_j обрабатывается состояние $\langle A \rightarrow X_1, \dots X_p, X_{p+1}, \dots X_m, m, [i, j] \rangle$, т. е. нетерминал A полностью разобран на интервале $[j, j]$. Для каждого состояния в dp_i вида $\langle B \rightarrow Y_1, \dots Y_t, A, Y_{t+2}, \dots Y_l, t, [k, i] \rangle$ в dp_j добавляют состояния $\langle B \rightarrow Y_1, \dots Y_t, A, Y_{t+2}, \dots Y_l, t + 1, [k, j] \rangle$. Таким образом, операция завершения разбора правила изменяет отметку p в правилах для нетерминала B и увеличивает интервал действия до j .

В худшем случае время работы алгоритма Эрли оценивается как $O(n^3)$ при длине предложения n ; для многих однозначных грамматик на практике наблюдается поведение ближе к $O(n^2)$ [19, 23].

1.3.3 Сравнение методов построения систем составляющих

Для сравнения алгоритмов СУК и Эрли выделены следующие критерии:

- вычислительная сложность алгоритмов в худшем случае;
- необходимость предварительного приведения к нормальной форме Хомского;
- максимальная длина правой части правила в грамматике;
- число параметров необходимых для разбора подотрезка предложения;
- число типов операций для построения составляющей;
- оценка числа состояний в худшем случае;
- оценка необходимой памяти в худшем случае;
- возможность построения span-таблицы синтаксических единиц предложения;
- возможность вероятностного выбора синтаксической единицы для разрешения неоднозначности разбора.

Сравнение в соответствии с сформулированными критериями приведено в таблице 1.2.



Таблица 1.2 — Сравнение алгоритмов СΥК и Эрли

Критерий	Алгоритм СΥК	Алгоритм Эрли	Примечание
Вычислительная сложность в худшем случае	$O(n^3)$	$O(n^3)$	На основе [23, 24]
Требуется предварительное приведение к НФХ	Да	Нет	Вид правил алгоритма СΥК: $A \rightarrow BC$ или $A \rightarrow w$
Максимальная длина правой части правила без преобразования	2	Не ограничена	
Число параметров, определяющих разбор подотрезка предложения	2	3	Алгоритм СΥК: (i, j) , $0 \leq i < j \leq n$ Алгоритм Эрли: $(r, p, [i, j])$
Число типов операций для построения составляющей	1	3	
Оценка числа состояний, худший случай	$O(n^2 N)$	$O(n^3 P)$	$ N $ — число различных нетерминалов в грамматике $ P $ — число правил в грамматике
Оценка памяти в худшем случае	$O(n^2)$	$O(n^3)$	На основе [19, 22, 23]
Построение span-таблицы всех синтаксических единиц предложения	Да	Нет	
Возможность вероятностного выбора синтаксической единицы для разрешения неоднозначности разбора	Да	Да	

В соответствии с представленным сравнением алгоритм СΥК является более предпочтительным для реализации метода построения деревьев составляющих для ограниченного естественного русского языка в связи с:

- более полным соответствием формализму деревьев составляющих, как множеству непересекающихся, либо вложенных подотрезков предложения;
- меньшими затратами памяти, числом состояний и параметров;
- возможностью построения span-таблицы всех синтаксических единиц предложения с целью дальнейшего анализа их употребимости.

1.4 Постановка задачи

Для реализации метода построения синтаксических деревьев ограниченного естественного русского языка необходимо определить:

- определить формат входных и выходных данных;
- определить правила грамматики применяемые в ограниченном русском языке;
- определить типы согласования применяемые в ограниченном русском языке;
- определить необходимые морфологические характеристики словоформ.

Метод построения синтаксических деревьев для ограниченного естественного русского

меченное дерево составляющих, вершины которого содержат метки синтаксических единиц в вершинах имеющих потомков, словоформы в листовых вершинах.

Наиболее употребимые в ограниченном естественном русском языке правила грамматики [18,25] представлены в приложении А. Наиболее употребимые [18,25] правила согласования приведены в приложении Б. Для использования правил согласования и грамматики необходимы следующие морфологические характеристики:

- существительное: род, число, падеж;
- прилагательное и причастие: род, число, падеж;
- глагол в личной форме: время, число, лицо, переходность;
- глагол в инфинитиве: переходность;
- местоимение: род, число, падеж, лицо;
- числительное: тип, падеж, число, род.

Для наречий, деепричастий, предлогов, союзов, частиц определяется только часть речи. При этом для предлогов необходимо указать падеж, который должно принимать существительное, согласующееся с данным предлогом.

Формализованная постановка задачи в виде функциональной модели в нотации IDEF0 представлена на рисунках 1.7, 1.8.

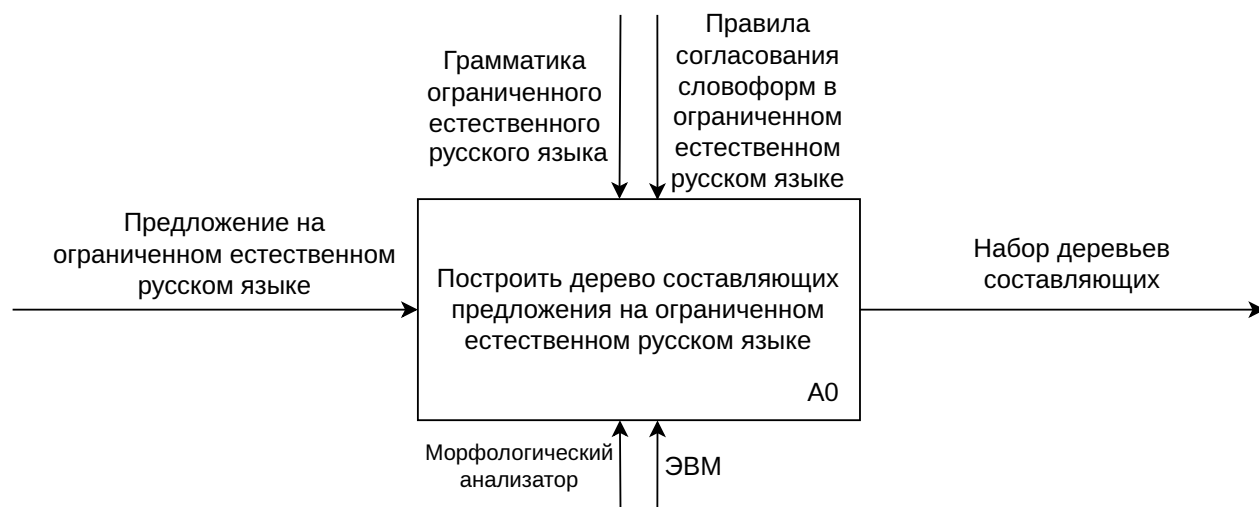


Рисунок 1.7 — IDEF0-диаграмма уровня A0

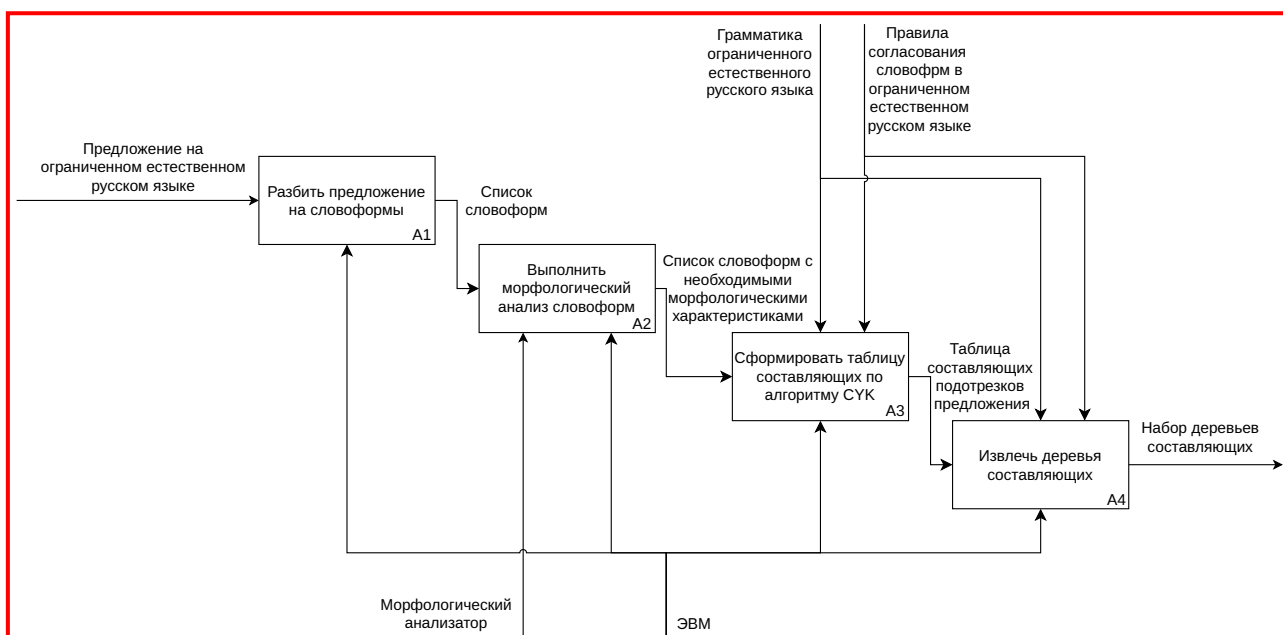


Рисунок 1.8 — IDEF0-диаграмма уровня A1

Выводы

В данном разделе введены основные понятия предметной области автоматизированной синтаксической разметки ограниченного естественного русского языка, описаны системы синтаксической разметки. Приведено сравнение данных систем. Описаны существующие методы построения систем составляющих, приведено их сравнение. Определены формат входных и выходных данных метода построения деревьев составляющих, описаны наиболее употребимые правила грамматики и согласования, выделены необходимые для их использования морфологические характеристики словоформ. Приведена формализованная постановка задачи в нотации IDEF0.

2 Конструкторский раздел

2.1 Требования к разрабатываемому методу

Разрабатываемый метод построения деревьев составляющих для ограниченного естественного русского языка должен удовлетворять следующим требованиям:

- 1) возможность работы с грамматикой не приведенной к НФХ — метод должен успешно обрабатывать правила вида $A \rightarrow BCD$ и $A \rightarrow B$, где в правой части правил располагаются нетерминалы, распространенные в грамматике русского языка, но не удовлетворяющие НФХ;
- 2) возможность обработки морфологической омонимии — в случае наличия нескольких вариантов морфологического разбора метод должен обработать каждый из них;
- 3) возможность учета правил согласования — существующие методы работают с грамматикой без учета правил согласования, поскольку данные правила мало распространены в романской и германской языковых группах для которых эти методы разрабатывались; в то же время в русском языке правила согласования важны для грамматически правильного синтаксического анализа [26, 27];
- 4) возможность определения признаков нетерминальной синтаксической единицы — метод должен определять морфологические признаки синтаксической единицы на основе морфологических признаков единиц-потомков;
- 5) возможность обработки пунктуации — метод должен обрабатывать знаки пунктуации внутри предложения, в связи с их ролью в обособлении синтаксических единиц;
- 6) возможность получения всех возможных деревьев составляющих — метод должен предоставлять все возможные варианты выделения синтаксических единиц в рамках предложения.

Для удовлетворения перечисленным требованиям, разработаны следующие решения:

- 1) приведение правил грамматики к правилам удовлетворяющим НФХ в ходе работы метода, путем ввода дополнительных вспомогательных нетерминалов, которые в дальнейшем удаляются из результирующего дерева составляющих; потомки таких нетерминальных вершин, становятся потомками предка нетерминала;
- 2) внесение на начальном этапе формирования таблицы составляющих по СΥК всех возможных наборов морфологических характеристик каждой словоформы;
- 3) объединение синтаксических единиц в ходе заполнения таблицы составляющих не только по правилам грамматики, но и правилам согласования, перечисленным в приложении В;
- 4) определение морфологических характеристик не листовой вершины в дереве составляющих по морфологическим характеристикам ее потомков на основе вида синтаксической единицы, которой соответствует вершина, и правил согласования, перечисленных в



приложении В;

5) обработка знаков пунктуации как отдельных терминалов;

6) рекурсивное восстановление дерева составляющих по полученной таблице составляющих с сохранением всех вариантов.

Разрабатываемый метод имеет следующие ограничения:

— работа с предложениями на ограниченном естественном русском языке, который задается наборами правил грамматики и согласования, описанными в приложениях Б и В соответственно;

— предложения не удовлетворяющие описанным правилам грамматики или согласования не обрабатываются;

— невозможность полноценной обработки непроективных конструкций;

— к обрабатываемым знакам пунктуации относятся запятые, двоеточия, точки с запятой; прочие знаки пунктуации не обрабатываются методом.

2.2 Используемые структуры и форматы данных

Для обработки данных разрабатываемым методом, необходимо выбрать используемые им структуры и форматы представления данных. В данном разделе описываются представления данных с которыми работает метод на различных этапах и используемые для этого структуры данных.

Для представления входной токенизированной последовательности перед обработкой морфологическим анализатором используется массив строк, содержащий словоформы входного предложения и обрабатываемые знаки препинания. Такое представление позволяет за линейное время обработать все словоформы предложения.

Последовательность словоформ получивших морфологические характеристики, а также обрабатываемые знаки препинания, представляется в виде массива пар, в которых первый элемент — непосредственно словоформа, второй — варианты ее морфологического разбора. Варианты морфологического разбора представляются в виде массива ассоциативных массивов, в которых ключами являются названия морфологических характеристик, значениями — значения морфологических характеристик. Такой подход позволяет обработать все возможные морфологические разборы за линейное время, а получение в конкретном разборе определенной характеристики осуществляется за $O(1)$ при реализации ассоциативного массива с помощью хеш-таблицы [28].

Таблица составляющих с которой работает алгоритм СΥΚ представлена двумерным массивом, каждая ячейка которого — ассоциативный массив, в котором ключ — название синтаксической единицы, значение — множество возможных наборов морфологических характеристик данной единицы. Такой подход позволяет для каждого подотрезка входного предложения сохранить более одной возможной синтаксической единицы, а для каждой такой единицы в



свою очередь более одного варианта набора морфологических характеристик. Множественные варианты необходимы вследствие морфологической и синтаксической омонимии, которые приводят к различным вариантам разборов подотрезков данного отрезка и, в простейшем случае, к различным вариантам разбора одной словоформы. Множество наборов морфологических характеристик в свою очередь состоит из неизменяемых множеств пар строковых значений, определяющих название и значение морфологических характеристик. Данное представление позволяет для каждой синтаксической единицы поддерживать изменяемое множество уникальных неизменяемых наборов морфологических характеристик во избежание повторяющихся наборов таких характеристик и проверки наличия определенного набора среди уже имеющихся за $O(1)$. При этом каждый набор представлен неизменяемым множеством, чтобы не допустить дублирование характеристик в одном наборе.

Грамматика представляется в виде ассоциативного массива, ключем в котором является название синтаксической единицы (нетерминал), значением — массив возможных разборов данной единицы. Каждый возможный разбор представляется массивом названий синтаксических единиц-потомков нетерминала-ключа. Такое представление позволяет за $O(1)$ определить потомков нетерминала. Однако, поскольку алгоритм СΥΚ производит разбор снизу-вверх [19], данное представление следует инвертировать, для получения нетерминала, который может быть составлен из двух данных синтаксических единиц. Таким образом, в инвертированном представлении грамматика является ассоциативным массивом, в котором ключами являются пары синтаксических единиц, а значениями — массив всех выводимых из них нетерминалов.

Для представления получаемого в результате работы метода дерева используется формат используется ассоциативный массив содержащий в качестве значений:

- имя синтаксической единицы соответствующей данной вершине;
- ассоциативный массив такой же структуры, описывающий потомков данной вершины;
- словоформу, соответствующую данной вершине, если она является листовой.

Для хранения грамматики грамматических правил и получаемых деревьев синтаксического разбора могут быть использованы файлы формата JSON, так как они соответствуют структуре ассоциативного массива, могут быть сериализованы в такие массивы и десериализованы из них средствами большинства языков программирования [29].

2.3 Описание ключевых этапов метода

С учетом сформулированных требований, были разработаны алгоритм разбиения входного предложения на словоформы и выделения обрабатываемых знаков препинания, схема которого представлена на рисунке 2.1.

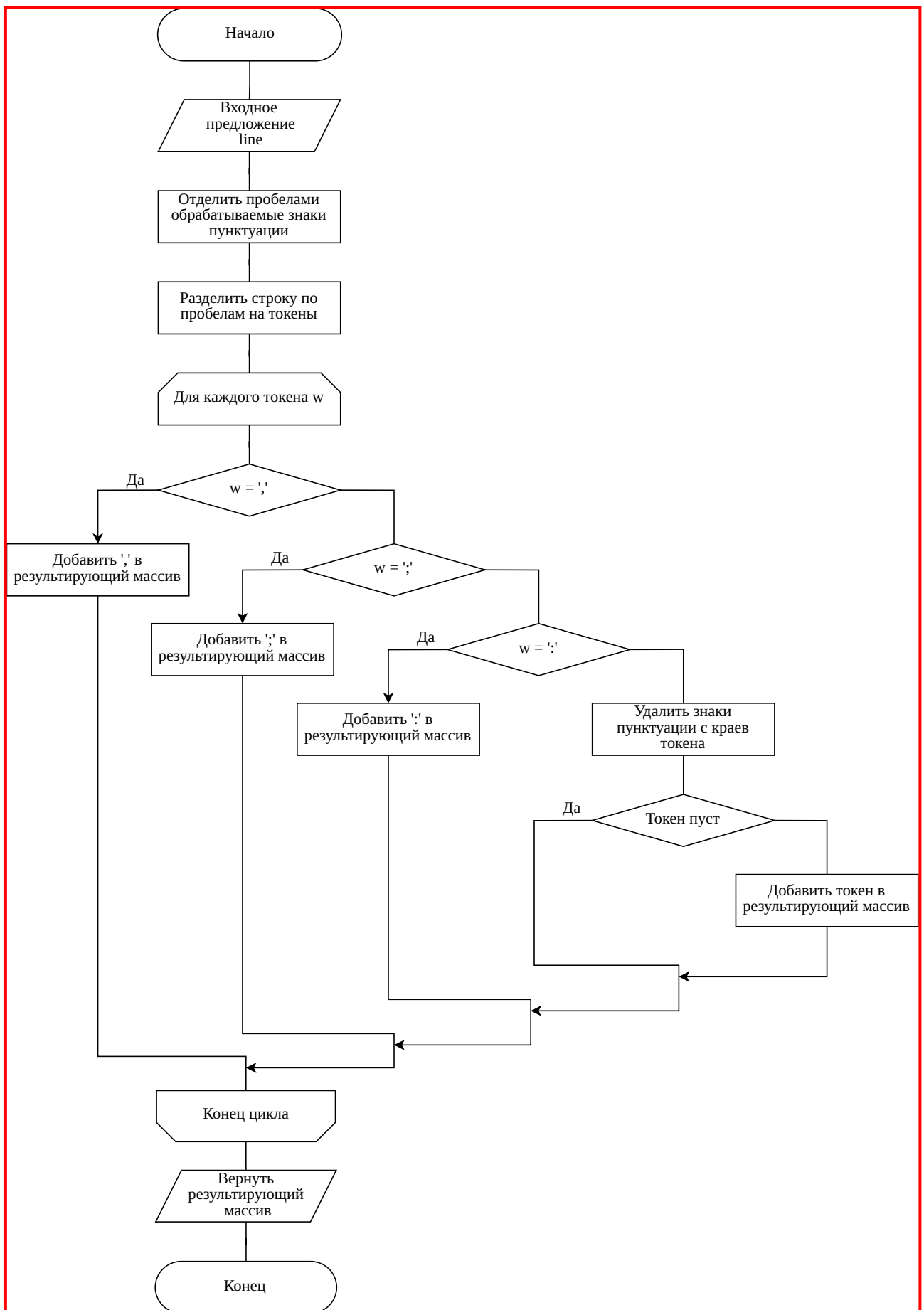


Рисунок 2.1 — Схема алгоритма разбиения входного предложения на словоформы и знаки препинания

Для получения морфологических характеристик словоформ и определения типов знаков препинания в полученной последовательности был разработан алгоритм, схема которого представлена на рисунке 2.2.

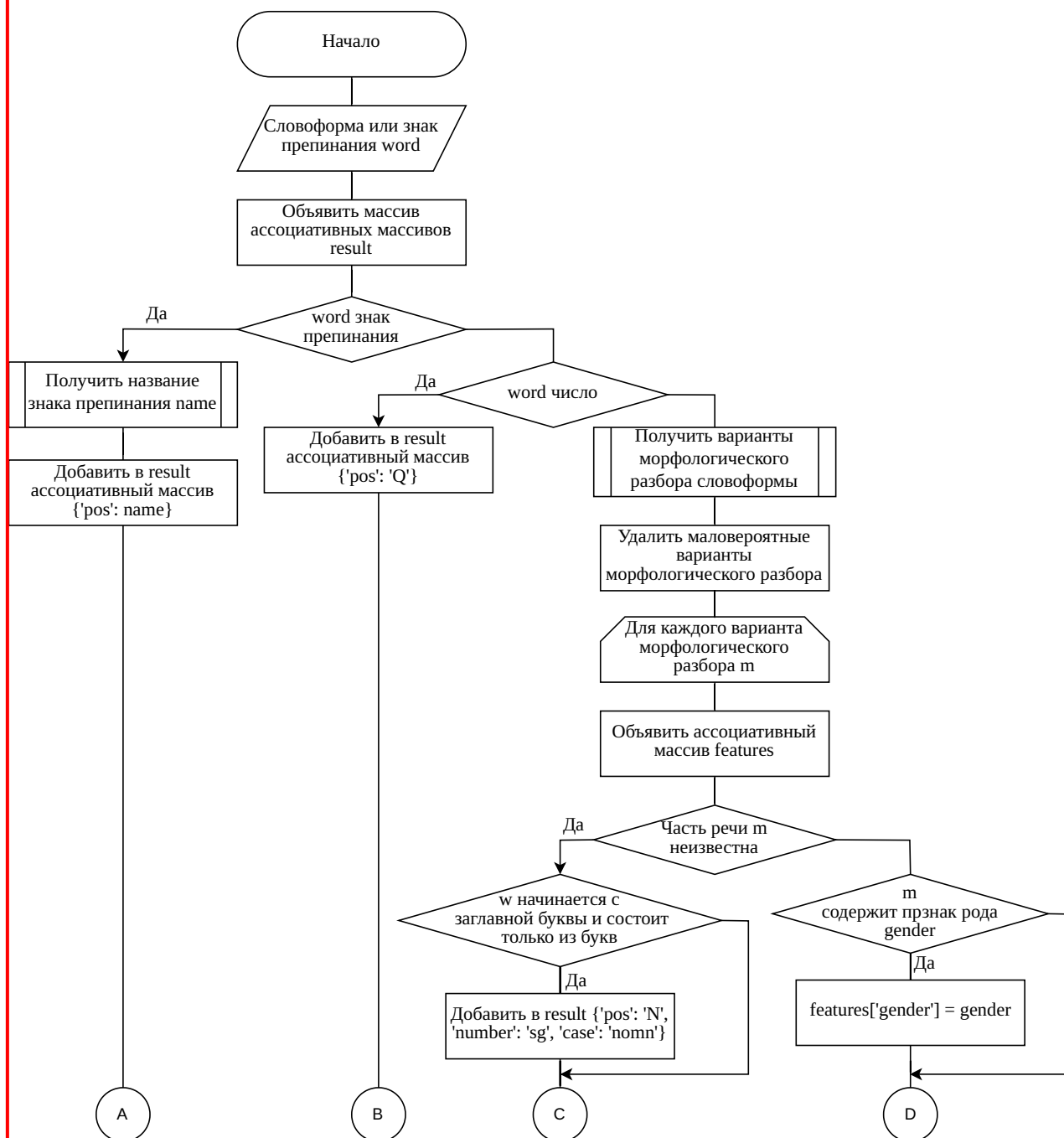


Рисунок 2.2 — Схема алгоритма получения морфологических характеристик словоформ и определения типов знаков препинания

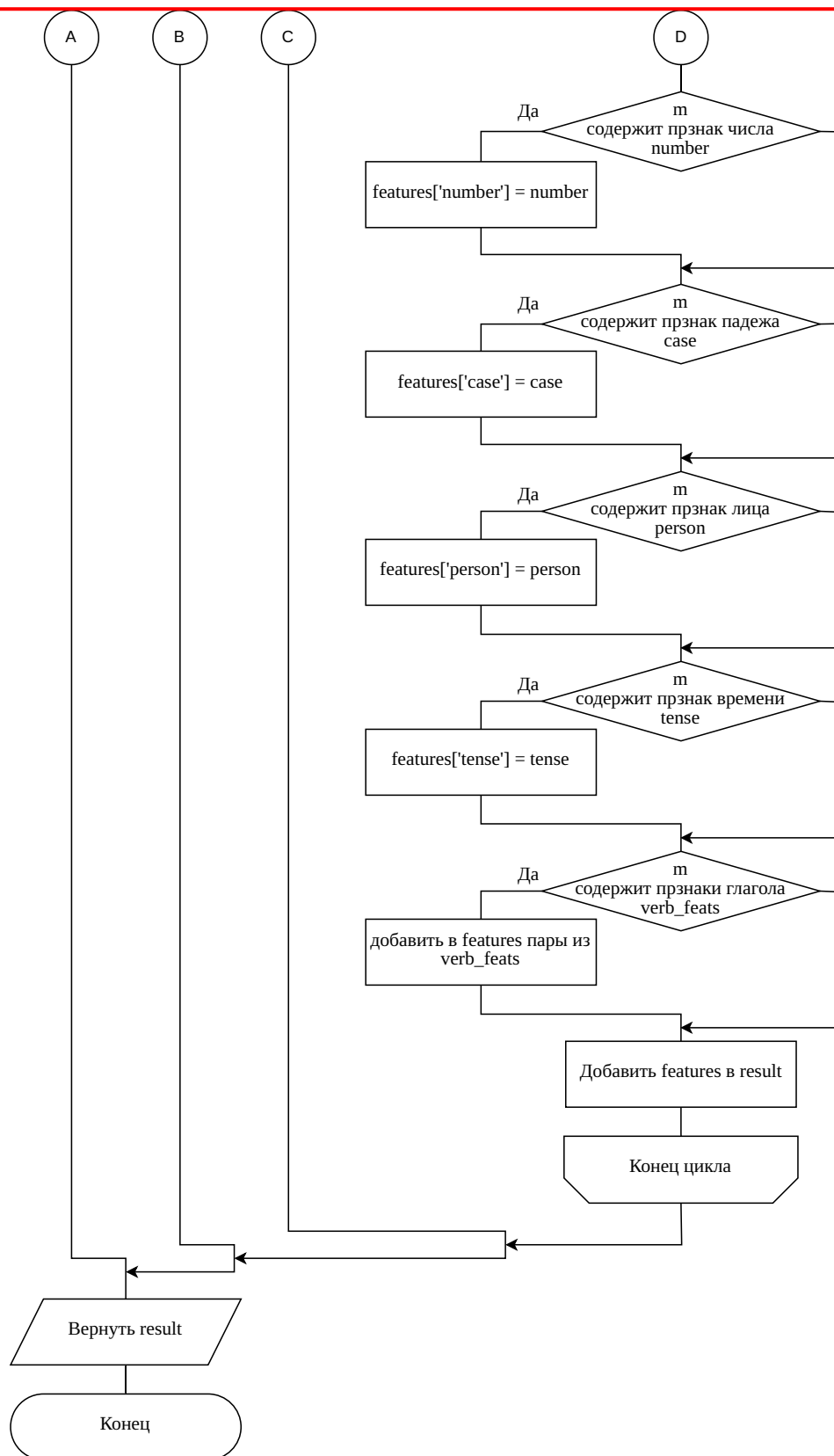
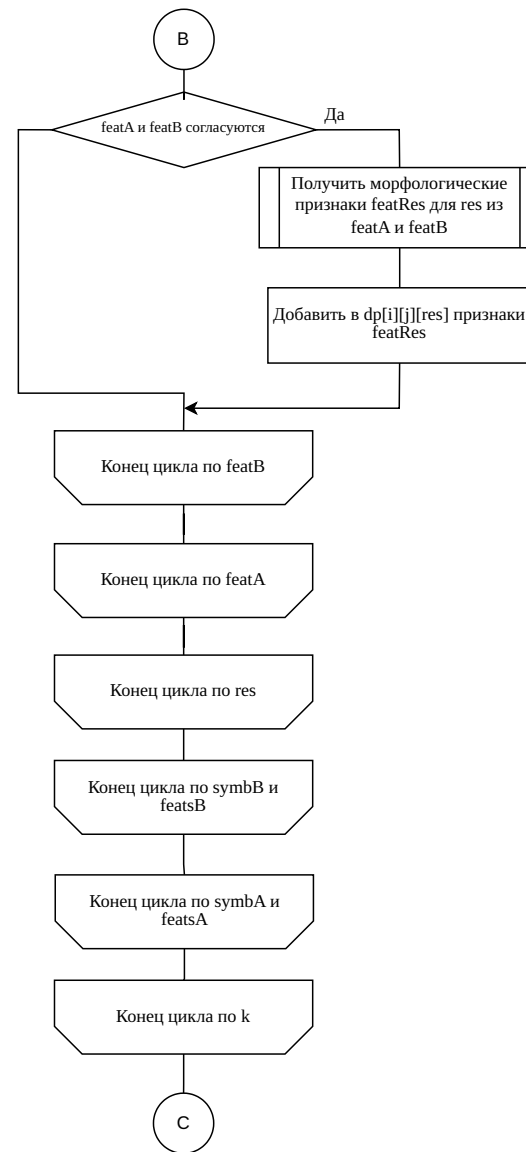
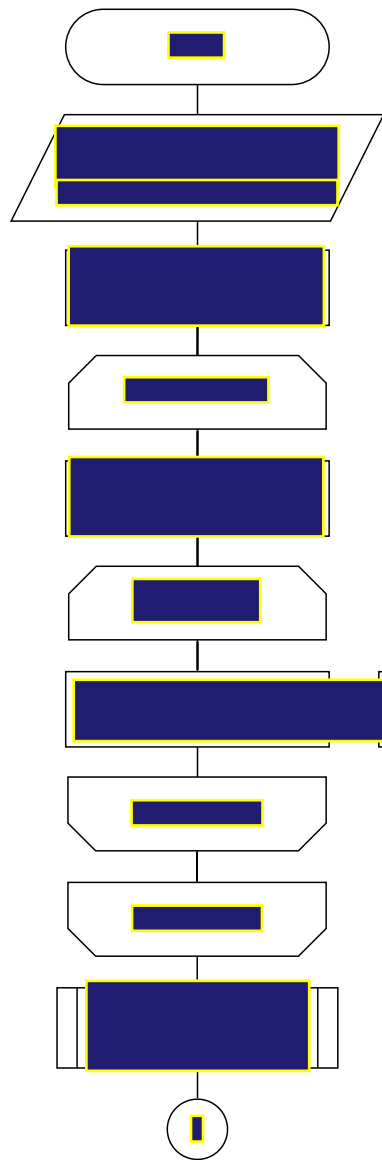
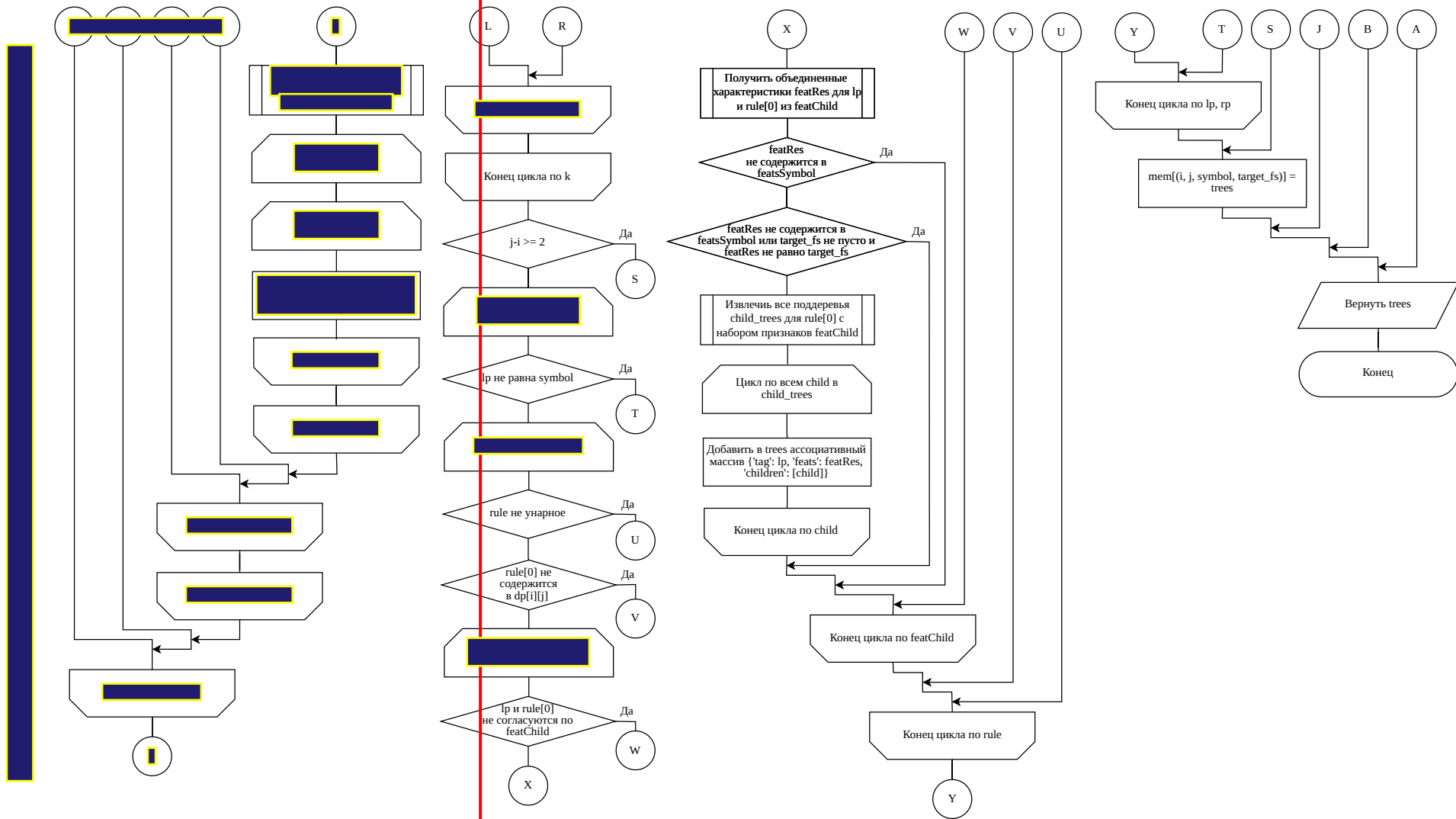


Рисунок 2.2 — Схема алгоритма получения морфологических характеристик словоформ и определения типов знаков препинания

Схема алгоритма заполнения таблицы составляющих приведена на рисунке 2.3. Данный алгоритм дополнительно использует проверку согласования и вывод нетерминалов из унарных





Разрабатываемое ПО включает в себя следующие компоненты:

- модуль морфологической предобработки — инкапсулирует разбиение входного предложения на токены и присваивание им морфологических характеристик;
- модуль синтаксического анализа — производит заполнение таблицы составляющих и построение синтаксического дерева;
- синтаксический анализатор — инкапсулирует работу с модулями морфологического и синтаксического анализа;
- модуль постобработки — формирует изображения и скобочной записи построенных деревьев составляющих;
- пользовательский интерфейс — предоставляет пользователю возможность строить деревья составляющих для вводимых предложений и просматривать ранее построенные деревья составляющих.

Диаграмма компонентов ПО, представлена на рисунке 2.5.

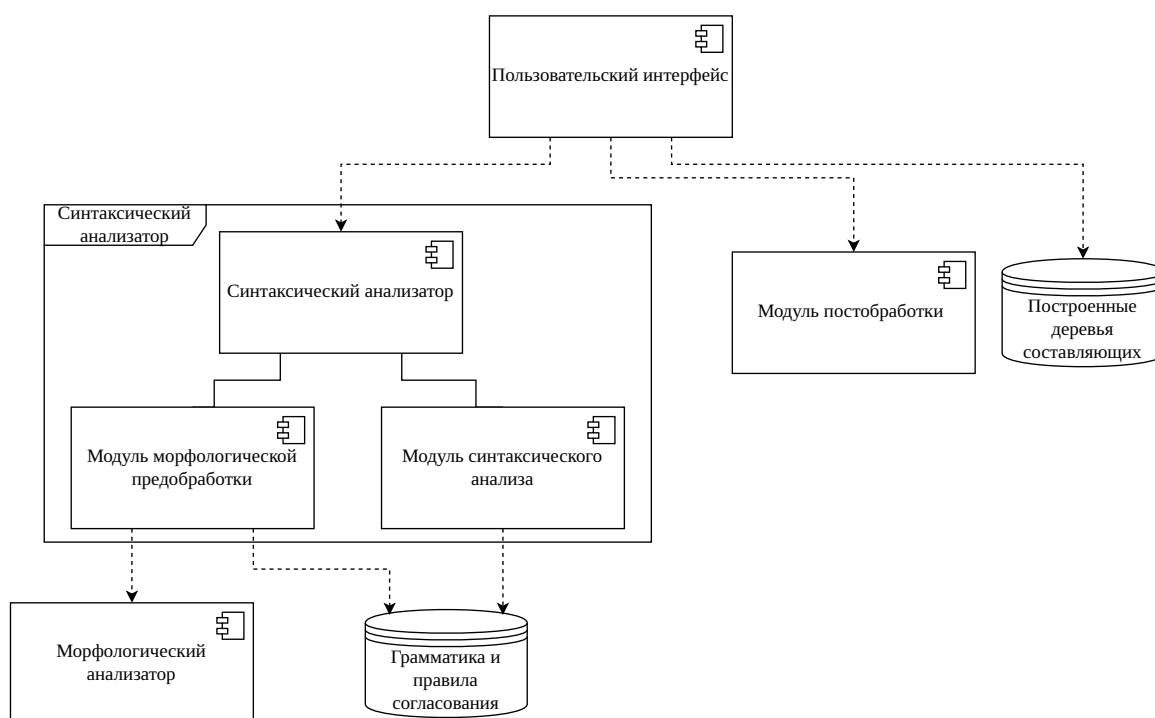


Рисунок 2.5 — Диаграмма компонентов разрабатываемого ПО

Выводы

В данном разделе были сформулированы требования и ограничения разрабатываемого метода построения синтаксических деревьев составляющих ограниченного естественного русского языка. Описаны используемые структуры и форматы данных, обоснован их выбор.

3 Технологический раздел

3.1 Выбор средств реализации

Наиболее важным внешним компонентом в разрабатываемом методе является морфологический анализатор. Предпочтительным для использования в разработанном методе является анализатор, основанный на словарном подходе, поскольку анализаторы данного типа не требуют обработки контекста. Среди анализаторов основанных на словарном подходе можно выделить следующие:

- MyStem [30] — морфологический анализатор разработанный И. Сегаловичем, реализованный на языке Си и доступный в качестве утилиты командной строки и подключаемой библиотеки для C/C++ и Python;
- Stemka [31] — морфологический анализатор разработанный А. Коваленко, реализованный на языке C++, доступный в качестве утилиты командной строки и подключаемой библиотеки для C++;
- xMorphy [32] — морфологический анализатор реализованный на языке C++ и доступный в качестве утилиты командной строки и подключаемой библиотеки для C++;
- PyMorphy [33] — морфологический анализатор, реализованный на языке Python и доступный в качестве подключаемой библиотеки для данного языка.

Для сравнения приведенных морфологических анализаторов были сформулированы следующие критерии:

- доступность в качестве подключаемой библиотеки;
- возможность получения необходимых морфологических характеристик для каждой из рассматриваемых частей речи;
- возможность получения нескольких вариантов морфологических характеристик.

Сравнение перечисленных морфологических анализаторов по сформулированным критериям приведено в таблице 3.1.

Таблица 3.1 — Сравнение систем синтаксической разметки

Критерий	MyStem	Stemka	xMorphy	PyMorphy
Использование в качестве подключаемой библиотеки	Да	Да	Да	Да
Получение необходимых морфологических характеристик	Да	Нет	Да	Да
Получение нескольких возможных вариантов морфологических характеристик	Нет	Да	Да	Да



На основании приведенного сравнения анализаторы xMorphy и PyMorphy являются наиболее подходящими для использования в разработанном методе, как наиболее полно удовлетворяющие требованиям.

В качестве языка реализации метода был выбран Python [34], поскольку данный язык программирования поддерживает реализации необходимых структуры данных, предоставляет возможность использования web-фреймворков для реализации серверной части web-приложения и совместим с выбранным в качестве одного из наиболее подходящих для использования в реализуемом методе морфологическим анализатором PyMorphy3 [33].

3.2 Пользовательский интерфейс

Пользовательский интерфейс представлен web-приложением, позволяющим:

- ввести предложение на русском языке;
- получить представление дерева зависимостей в одном из следующих графических или текстовых форматов:
 - изображение дерева в формате .png;
 - текстовое представление дерева в формате JSON;
 - текстовое представление в виде скобочной последовательности;
 - текстовое представление в формате DOT.

Для реализации серверной составляющей web-приложения был использован язык Python и фреймворк FastAPI [35]. Для реализации клиентской составляющей web-приложения был использован язык TypeScript [36] и фреймворк React [37]. Пример пользовательского web-интерфейса представлен на рисунке 3.1.

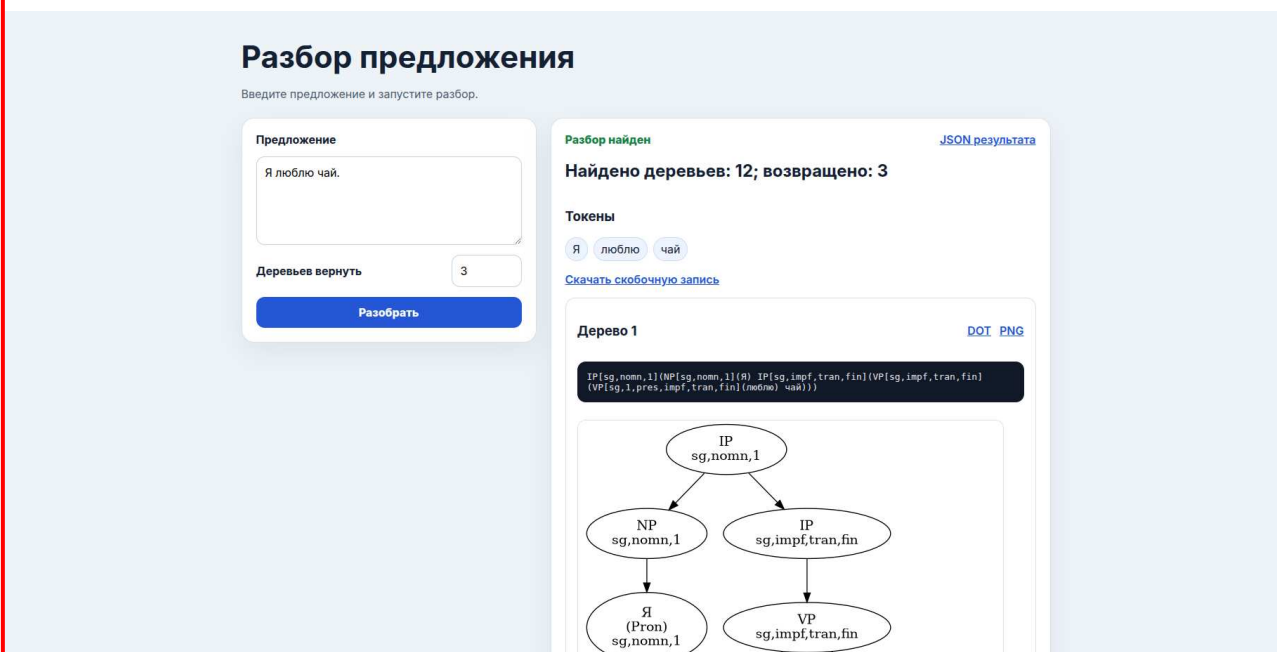


Рисунок 3.1 — Пример пользовательского интерфейса web-приложения

3.3 Реализация ключевых этапов метода

Реализация алгоритма получения морфологических характеристик словоформ и определения типов знаков препинания приведена в листинге 3.1.

Листинг 3.1 – Реализация алгоритма получения морфологических характеристик словоформ и типов знаков препинания

```
1 def get_features(word):
2     punct_feats = _get_punctuation_features(word)
3     if punct_feats is not None:
4         return punct_feats
5
6     digit_feats = _get_digit_features(word)
7     if digit_feats is not None:
8         return digit_feats
9
10    parses = morph.parse(word)
11    filtered = [p for p in parses if p.score >= MIN_PARSE_SCORE]
12    if not filtered:
13        filtered = parses[:1]
14
15    results = []
16    for p in filtered:
17        feats = {'pos': POS_MAP.get(p.tag.POS, 'X')}
18        g = p.tag.grammemes
19
20        gender = _extract_gender(g)
21        if gender is not None:
22            feats['gender'] = gender
23        number = _extract_number(g)
24        if number is not None:
25            feats['number'] = number
26        case = _extract_case(g)
27        if case is not None:
28            feats['case'] = case
29        person = _extract_person(g)
30        if person is not None:
31            feats['person'] = person
32        tense = _extract_tense(g)
33        if tense is not None:
```



```

34         feats['tense'] = tense
35
36         verb_feats = _extract_verb_features(g, p.tag.POS)
37         feats.update(verb_feats)
38
39         results.append(feats)
40
41     if len(results) == 1 and results[0].get("pos") == "X":
42         core = word.replace("ë", "e").replace("Ë", "E")
43         if len(core) >= 2 and core[0].isupper() and core.replace("-",
44             ↪ " ").isalpha():
45             return [{"pos": "N", "number": "sg", "case":
46                 ↪ "nomn"}]
47
48     return results

```

В листинге 3.2 приведена реализация алгоритма построения таблицы составляющих. Данная реализация использует функцию вывода всех нетерминалов по унарным правилам, реализация которой представлена в листинге 3.3.

Листинг 3.2 – Реализация алгоритма построения таблицы составляющих

```

1 def build_cyk_table(token_feature_pairs, unary_index, binary_index):
2     n = len(token_feature_pairs)
3     tokens = [pair[0] for pair in token_feature_pairs]
4     dp = [[dict() for _ in range(n+1)] for _ in range(n+1)]
5
6     for j in range(1, n+1):
7         token, features_list = token_feature_pairs[j-1]
8         for feat in features_list:
9             pos = feat['pos']
10            feat_set = frozenset(feat.items())
11            dp[j-1][j].setdefault(pos, set()).add(feat_set)
12            for lhs in unary_index.get(pos, []):
13                dp[j-1][j].setdefault(lhs, set()).add(feat_set)
14
15    while _unary_closure_round(dp, unary_index, tokens, n):
16        pass
17
18    for length in range(2, n + 1):
19        for i in range(0, n - length + 1):

```



```

20         j = i + length
21         for k in range(i + 1, j):
22             left_cell = dp[i][k]
23             right_cell = dp[k][j]
24             for symA, featsA_set in left_cell.items():
25                 for symB, featsB_set in right_cell.items():
26                     for lhs in binary_index.get((symA, symB), []):
27                         for featA in featsA_set:
28                             for featB in featsB_set:
29                                 if agreement_check(
30                                     lhs, symA, featA, symB, featB,
31                                     tokens=tokens,
32                                     span_left=(i, k),
33                                     span_right=(k, j),
34                                 ):
35                                     new_feat = merge_features(lhs, symA,
36                                         ↪ featA, symB, featB)
37                                     dp[i][j].setdefault(lhs,
38                                         ↪ set()).add(new_feat)
39         while _unary_closure_round(dp, unary_index, tokens, n):
40             pass
41     return dp

```

Листинг 3.3 – Реализация алгоритма добавления всех новых нетерминалов по унарным правилам

```

1 def _unary_closure_round(dp, unary_index, tokens, n):
2     changed = False
3     for length in range(1, n + 1):
4         for i in range(0, n - length + 1):
5             j = i + length
6             for sym, feat_set in list(dp[i][j].items()):
7                 for lhs in unary_index.get(sym, []):
8                     for feat in feat_set:
9                         if not agreement_check_unary(lhs, sym, feat):
10                             continue
11                         new_feat = merge_features_unary(lhs, sym, feat)
12                         bucket = dp[i][j].setdefault(lhs, set())
13                         before = len(bucket)
14                         bucket.add(new_feat)

```

```
15             if len(bucket) > before:
16                 changed = True
17     return changed
```

Реализация алгоритма восстановления дерева составляющих по таблице составляющих представлена в листинге 3.4.

Листинг 3.4 – Реализация алгоритма восстановления дерева составляющих

```
1 def extract_trees(i, j, symbol, word_chain, dp, grammar, memo,
    ↪ target_feat=None):
2     feats_in_cell = dp[i][j].get(symbol, set())
3     if target_feat is not None and target_feat not in feats_in_cell:
4         return []
5
6     memo_key = (i, j, symbol, target_feat)
7     if memo_key in memo:
8         return memo[memo_key]
9
10    trees = []
11
12    if j - i == 1:
13        if symbol in TERMINAL_TAGS:
14            word = word_chain[i]
15            for fs in feats_in_cell:
16                if target_feat is not None and fs != target_feat:
17                    continue
18                feats_dict = dict_from_frozenset(fs)
19                trees.append({
20                    'tag': symbol,
21                    'feats': feats_dict,
22                    'word': word
23                })
24    for lhs, rules in grammar.items():
25        if lhs != symbol:
26            continue
27        for rhs in rules:
28            if len(rhs) != 1:
29                continue
30            child_sym = rhs[0]
```

```

31         for fs in feats_in_cell:
32             if target_feat is not None and fs != target_feat:
33                 continue
34             child_trees = extract_trees(i, j, child_sym, word_chain, dp,
35                                         ↪ grammar, memo, target_feat=fs)
36             for child in child_trees:
37                 feats_dict = dict_from_frozenset(fs)
38                 trees.append({
39                     'tag': symbol,
40                     'feats': feats_dict,
41                     'children': [child]
42                 })
43             memo[memo_key] = trees
44         return trees
45     for k in range(i+1, j):
46         left_cell = dp[i][k]
47         right_cell = dp[k][j]
48         for lhs, rules in grammar.items():
49             if lhs != symbol:
50                 continue
51             for rhs in rules:
52                 if len(rhs) != 2:
53                     continue
54                 A, B = rhs[0], rhs[1]
55                 if A not in left_cell or B not in right_cell:
56                     continue
57                 for featA in left_cell[A]:
58                     for featB in right_cell[B]:
59                         if not agreement_check(
60                             lhs,
61                             A,
62                             featA,
63                             B,
64                             featB,
65                             tokens=word_chain,
66                             span_left=(i, k),
67                             span_right=(k, j),
68                             ):

```



```

69         continue
70         merged = merge_features(lhs, A, featA, B, featB)
71         if merged not in feats_in_cell:
72             continue
73         if target_feat is not None and merged != target_feat:
74             continue
75         left_trees = extract_trees(i, k, A, word_chain, dp,
76             ↪ grammar, memo, target_feat=featA)
77         right_trees = extract_trees(k, j, B, word_chain, dp,
78             ↪ grammar, memo, target_feat=featB)
79         for lt in left_trees:
80             for rt in right_trees:
81                 feats_dict = dict_from_frozenset(merged)
82                 trees.append({
83                     'tag': lhs,
84                     'feats': feats_dict,
85                     'children': [lt, rt]
86                 })
87
88 if j - i >= 2:
89     for lhs, rules in grammar.items():
90         if lhs != symbol:
91             continue
92         for rhs in rules:
93             if len(rhs) != 1:
94                 continue
95             child_sym = rhs[0]
96             if child_sym not in dp[i][j]:
97                 continue
98             for cfs in dp[i][j][child_sym]:
99                 if not agreement_check_unary(lhs, child_sym, cfs):
100                     continue
101                 merged = merge_features_unary(lhs, child_sym, cfs)
102                 if merged not in feats_in_cell:
103                     continue
104                 if target_feat is not None and merged != target_feat:
105                     continue
106                 child_trees = extract_trees(

```



```

105         i, j, child_sym, word_chain, dp, grammar, memo,
           ↪ target_feat=cfs
106     )
107     for child in child_trees:
108         feats_dict = dict_from_frozenset(merged)
109         trees.append(
110             {
111                 "tag": lhs,
112                 "feats": feats_dict,
113                 "children": [child],
114             }
115         )
116
117     memo[memo_key] = trees
118     return trees

```

3.4 Тестирование разработанного метода

Для тестирования корректности работы алгоритма построения таблицы составляющих были выделены классы эквивалентности входных данных, представленные в таблице 3.2.

Таблица 3.2 — Классы эквивалентности входных данных алгоритма построения таблицы составляющих

Описание класса эквивалентности	Пример входных данных, удовлетворяющих классу эквивалентности	Ожидаемые выходные данные
Пустая последовательность токенов	Пустой список <i>token_feature_pairs</i>	Пустая таблица составляющих <i>dp</i> , размерности 1×1
Последовательность токенов из одного элемента, для обработки унарными правилами, с единственным набором морфологических признаков	<i>token_feature_pairs</i> содержит единственную пару («кот», [{«pos»: «N», «number»: «sg», «case»: «nomn»}]), <i>unary</i> = {«N»: [«NP»], «NP»: [«IP»]}	Таблица составляющих <i>dp</i> размерности 2×2 . В <i>dp</i> [0][1] содержится ключ <i>N</i> и ключи <i>NP</i> , <i>IP</i> полученные выводом из унарных правил, с соответствующими наборами морфологических признаков. <i>dp</i> [0][0], <i>dp</i> [1][1] — пусты

Продолжение таблицы 3.2 — Классы эквивалентности входных данных алгоритма построения таблицы составляющих

Описание класса эквивалентности	Пример входных данных, удовлетворяющих классу эквивалентности	Ожидаемые выходные данные
Последовательность токенов из одного элемента, для обработки унарными правилами, с несколькими наборами морфологических признаков	<i>token_feature_pairs</i> содержит единственную пару («стали», [{«pos» : «N», «number» : «sg», «case» : «gent»}, {«pos» : «V», «number» : «pl», «tense» : «past», «verb_form» : «fin»}])	Таблица составляющих <i>dp</i> размерности 2×2 . В <i>dp</i> [0][1] содержатся ключи <i>N</i> , <i>V</i> с соответствующими морфологическими характеристиками. <i>dp</i> [0][0], <i>dp</i> [1][1] — пусты
Последовательность токенов, из которой выводится корень дерева составляющих, посредством унарных правил	<i>token_feature_pairs</i> содержит единственную пару («бежим», [{«pos» : «V», «number» : «sg», «verb_form» : «fin»}]) <i>unary</i> = {«V» : [«VP»], «VP» : [«IP»]}	Таблица составляющих <i>dp</i> размерности 2×2 . В <i>dp</i> [0][1] содержатся ключи <i>V</i> , <i>VP</i> и <i>IP</i> с соответствующими морфологическими характеристиками. <i>dp</i> [0][0], <i>dp</i> [1][1] — пусты
Последовательность согласованных токенов из которой выводится корень дерева составляющих, посредством бинарных правил	<i>token_feature_pairs</i> содержит пары («ком», [{«pos» : «N», «number» : «sg», «case» : «nomn», «gender» : «m»}]), («сnum», [{«pos» : «V», «number» : «sg», «person» : 3, «tense» : «pres», «verb_form» : «fin»}]) <i>unary</i> = {«V» : [«VP»], «N» : [«NP»]} <i>binary</i> = {(«NP», «VP») : [«IP»]}	Таблица составляющих <i>dp</i> размерности 2×2 . В <i>dp</i> [0][2] содержится ключ <i>IP</i> , в <i>dp</i> [0][1] — ключ <i>NP</i> , в <i>dp</i> [1][2] — ключ <i>VP</i> , значения определяют морфологические характеристики соответствующих составляющих

Продолжение таблицы 3.2 — Классы эквивалентности входных данных алгоритма построения таблицы составляющих

Описание класса эквивалентности	Пример входных данных, удовлетворяющих классу эквивалентности	Ожидаемые выходные данные
Последовательность несогласованных токенов	<i>token_feature_pairs</i> содержит пары («ком», [{«pos» : «N», «number» : «sg», «case» : «nomn», «gender» : «m»}]), («снят», [{«pos» : «V», «number» : «pl», «person» : 3, «tense» : «pres», «verb_form» : «fin»}]) <i>unary</i> = {«V» : [«VP»], «N» : [«NP»]} <i>binary</i> = {(«NP», «VP») : [«IP»]}	Таблица составляющих <i>dp</i> размерности 3×3 . В <i>dp</i> [0][2] содержится ключ <i>IP</i> , в <i>dp</i> [0][1] — ключ <i>NP</i> , в <i>dp</i> [1][2] — ключ <i>VP</i> , значения определяют морфологические характеристики соответствующих составляющих
Последовательность согласованных токенов, описывающая вложенную структуру составляющих, из которой выводится корень дерева составляющих	<i>token_feature_pairs</i> содержит пары («кошка», [{«pos» : «N», «number» : «sg», «case» : «nomn», «gender» : «f»}]), («ест», [{«pos» : «V», «number» : «sg», «person» : 3, «tense» : «pres», «verb_form» : «fin», «trans» : «tran»}]), («рыбу», [{«pos» : «N», «number» : «sg», «case» : «accs», «gender» : «f»}]) <i>unary</i> = {«V» : [«VP»], «N» : [«NP»]} <i>binary</i> = {(«NP», «VP») : [«IP»], («V», «NP») : [«VP»]}	Таблица составляющих <i>dp</i> размерности 4×4 . В <i>dp</i> [0][1] и <i>dp</i> [2][3] содержатся ключи <i>NP</i> , в <i>dp</i> [0][3] и <i>dp</i> [0][2] — ключ <i>IP</i> , в <i>dp</i> [1][3] — ключ <i>VP</i> , значения определяют морфологические характеристики соответствующих составляющих

Также были выделены классы эквивалентности, представленные в таблице 3.3, для тестирования алгоритма восстановления дерева составляющих по получаемым таблицам.



Таблица 3.3 — Классы эквивалентности входных данных алгоритма восстановления дерева составляющих

Описание класса эквивалентности	Пример входных данных, удовлетворяющих классу эквивалентности	Ожидаемые выходные данные
Терминал на отрезке длины 1	$i = 0, j = 1, symbol = \langle N \rangle,$ $word_chain = [\langle kom \rangle],$ $dp[0][1] = \{\langle N \rangle : \{\{\langle number \rangle : \langle sg \rangle\}\}\},$ $target_feat = None$	Список из одного узла-листа: $[\{\langle tag \rangle : \langle N \rangle,$ $\langle feats \rangle : \{\langle pos \rangle : \langle N \rangle,$ $\langle number \rangle : \langle sg \rangle\},$ $\langle word \rangle : \langle kom \rangle\}]$
Нетерминальный символ на отрезке длины 1, выведенный посредством унарного правила	$i = 0, j = 1, symbol = \langle NP \rangle,$ $word_chain = [\langle kom \rangle],$ $dp[0][1] = \{\langle N \rangle : \{\{\langle number \rangle : \langle sg \rangle\}\},$ $\langle NP \rangle : \{\{\langle number \rangle : \langle sg \rangle\}\},$ $grammar = \{\langle NP \rangle : [[\langle N \rangle]]\},$ $target_feat = None$	Список из одного узла с единственным потомком-листом: $[\{\langle tag \rangle : \langle NP \rangle,$ $\langle feats \rangle : \{\{\langle number \rangle : \langle sg \rangle\}\},$ $\langle children \rangle : [\{\langle tag \rangle : \langle N \rangle,$ $\langle feats \rangle : \{\langle number \rangle : \langle sg \rangle\},$ $\langle word \rangle : \langle kom \rangle\}]]]$
Соответствующий символ отсутствует в ячейке $dp[i][j]$	$i = 0, j = 1, symbol = \langle VP \rangle,$ $word_chain = [\langle kom \rangle],$ $dp[0][1] = \{\langle N \rangle : \{\{\langle number \rangle : \langle sg \rangle\}\}\},$ $target_feat = None$	Пустой список []
Символ в ячейке не согласуется с $target_feat$	$i = 0, j = 1, symbol = \langle N \rangle,$ $word_chain = [\langle kom \rangle],$ $dp[0][1] = \{\langle N \rangle : \{\{\langle number \rangle : \langle sg \rangle\}\}\},$ $target_feat = \{\langle V \rangle, \langle sg \rangle, \langle past \rangle\}$	Пустой список []
Токен с несколькими наборами морфологических признаков	$i = 0, j = 1, symbol = \langle N \rangle,$ $word_chain = [\langle стали \rangle],$ $dp[0][1] = \{\langle N \rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle case \rangle : \langle gent \rangle\},$ $\langle V \rangle : \{\{\langle number \rangle : \langle pl \rangle,$ $\langle tense \rangle : \langle past \rangle\}\}\},$ $target_feat = None$	Список из двух узлов-листьев: $[\{\langle tag \rangle : \langle N \rangle,$ $\langle feats \rangle : \{\langle pos \rangle : \langle N \rangle,$ $\langle number \rangle : \langle sg \rangle,$ $\langle case \rangle : \langle gent \rangle\},$ $\langle word \rangle : \langle стали \rangle\},$ $\{\langle tag \rangle : \langle V \rangle,$ $\langle feats \rangle : \{\langle pos \rangle : \langle V \rangle,$ $\langle number \rangle : \langle pl \rangle\},$ $\langle word \rangle : \langle стали \rangle\}]$

Продолжение таблицы 3.3 — Классы эквивалентности входных данных алгоритма восстановления дерева составляющих

Описание класса эквивалентности	Пример входных данных, удовлетворяющих классу эквивалентности	Ожидаемые выходные данные
Нетерминал на отрезке длины больше одного, выведенный посредством бинарного правила	$i = 0, j = 2, symbol = \langle VP \rangle,$ $word_chain = [\langle ect \rangle, \langle рыбу \rangle],$ $dp[0][1] = \{\langle V \rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle tense \rangle : \langle pres \rangle\}\}\},$ $dp[1][2] = \{\langle NP \rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle case \rangle : \langle accs \rangle\}\}\},$ $dp[0][2] = \{\langle VP \rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle tense \rangle : \langle pres \rangle\}\}\},$ $grammar = \{\langle VP \rangle : [[\langle V \rangle, \langle NP \rangle]],$ $\langle NP \rangle : [[\langle N \rangle]]\},$ $target_feat = None$	Список из одного узла VP с двумя потомками NP и V
Нетерминал на отрезке длины больше одного, выведенный посредством унарного правила	$i = 0, j = 2, symbol = \langle IP \rangle,$ $word_chain = [\langle ком \rangle, \langle cnum \rangle],$ $dp[0][2] = \{\langle VP \rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle tense \rangle : \langle pres \rangle\}\}\},$ $\langle IP \rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle gender \rangle : \langle m \rangle\}\}\},$ $grammar = \{\langle IP \rangle : [[\langle VP \rangle]],$ $\langle VP \rangle : [[\langle NP \rangle, \langle V \rangle]], \langle NP \rangle : [[\langle N \rangle]]\},$ $target_feat = None$	Список из одного узла IP с единственным потомком-узлом VP , у которого в свою очередь два потомка NP и V , у первого из которых один потомок N
Символ и значение совпадающее с $target_feat$ находится в мемоизации	Повторный вызов с кортежем $(i, j, symbol, target_feat)$	Значение $memo[(i, j, symbol, target_feat)]$ без повторного обхода dp и грамматики
Символ является вспомогательным нетерминалом для бинаризации небинарных отношений $\langle_AUX_i\rangle$	$i = 0, j = 2, symbol = \langle_AUX_i\rangle,$ $dp[0][2] = \{\langle_AUX_i\rangle : \{\{\langle number \rangle : \langle sg \rangle,$ $\langle tense \rangle : \langle pres \rangle\}\}\},$ $grammar$ содержит правило $\langle_AUX_i\rangle \rightarrow [\langle NP \rangle, \langle PP \rangle],$ $target_feat = None$	Список деревьев, узел которых имеет $\langle tag \rangle = \langle_AUX_i\rangle$ и двух потомков NP и PP

Все тесты, сформированные по выделенным классам эквивалентности, для алгоритмов построения таблицы составляющих и восстановления деревьев составляющих были пройдены. Покрытие функции *extract_trees* составило 90.6%, покрытие функции *build_cyk_table* — 97.6%.



Выводы

В данном разделе выбраны средства реализации, включая необходимые библиотеки и фреймворки, реализован разработанный метод и пользовательский интерфейс, представленный web-приложением. Приведен исходный код реализации ключевых этапов разработанного метода, проведено их тестирование.



4 Исследовательский раздел

4.1 Характеристики устройства и постановка исследования

Исследования проводились на машине со следующими характеристиками:

- процессор Intel(R) Core(TM) i5-10210U [38], тактовая частота 1.60 ГГц, 8 логических ядер;
- оперативная память: 16 ГБ;
- операционная система: Ubuntu 24.04 LTS [39].

Задачами данного исследования являются:

- получение зависимости времени выполнения ключевых этапов разработанного метода от числа словоформ во входном предложении;
- получение оценки точности и полноты разработанного метода.

Для получения зависимости времени выполнения ключевых этапов разработанного метода от числа словоформ в предложении, были проведены серии из 500 замеров для каждого числа n словоформ в предложении. В данном случае в число словоформ включаются знаки препинания, поскольку в разработанном методе они анализируются аналогично служебным членам предложения. Для проведения замеров были сформулированы 20 предложений, состоящие из $n = \overline{1, 20}$ словоформ. Верхний предел числа словоформ был выбран на основе среднего числа словоформ в современных художественных текстах равного 8 и 20 в юридических текстах [40].

Для оценки точности и полноты получаемых разработанным методом деревьев составляющих были использованы данные корпуса RuConst [7,41]. Основной единицей сравнения приведенных в корпусе деревьев составляющих и получаемых разработанным методом была выбрана тройка:

$$\langle l, i, j \rangle, \quad (4.1)$$

где l — метка составляющей;

i — индекс первой словоформы составляющей в исходном предложении;

j — индекс последней словоформы составляющей в исходном предложении.

метки, получаемые разработанным методом, отличные от перечисленных не учитываются в сравнении.

Используемый в качестве эталонного корпус содержит единственное дерево составляющих для каждого предложения. Для проведения сравнения, среди всех получаемых разработанным методом деревьев составляющих выбирается единственное. Число совпавших с эталонным деревом троек 4.1 максимально для выбранного дерева.

Таким образом, если G — множество троек вида 4.1 дерева составляющих эталонного корпуса, P — множество троек вида 4.1 выбранного для сравнения дерева, построенного разработанным методом, то полнота, точность, $F1$ -мера определяются следующим образом:



$$\text{Precision} = \begin{cases} \frac{TP}{FP + TP}, & TP + FP \neq 0 \\ 0, & TP + FP = 0 \end{cases}; \quad (4.2)$$

$$\text{Recall} = \begin{cases} \frac{TP}{FN + TP}, & TP + FN \neq 0 \\ 0, & TP + FN = 0 \end{cases}; \quad (4.3)$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (4.4)$$

где $TP = |G \cap P|$ — тройки 4.1, совпавшие в эталонном дереве составляющих и в полученном разработанном методом;

$FP = |P \setminus G|$ — тройки 4.1, присутствующие в полученном разработанном методом дереве составляющих, но отсутствующие в эталонном;

$FN = |G \setminus P|$ — тройки 4.1, присутствующие в эталонном дереве составляющих, но отсутствующие в полученном разработанном методом.

Результирующие значения точности, полноты и F1-меры вычисляются как среднее арифметическое по результатам выражений 4.2, 4.3 и 4.4 для 500 предложений корпуса [41], количество словоформ в которых не превосходит 15.

4.2 Результаты исследования

Зависимость медианного время построения таблицы составляющих от числа словоформ в исходном предложении представлена на рисунке 4.1



Рисунок 4.1 — Зависимость медианного времени выполнения алгоритма построения таблицы составляющих от числа словоформ во входном предложении

Полученные данные были аппроксимированы полиномом третьей степени:

$$T_{table}(n) \approx 7.85 \cdot 10^{-6} n^3 - 1.29 \cdot 10^{-4} n^2 + 1.01 \cdot 10^{-3} n - 1.56 \cdot 10^{-3} \quad (4.5)$$

, где n — число словоформ в предложении.

Коэффициенты полинома были получены по методу наименьших квадратов, степень полинома была определена с помощью информационного критерия Акаике [42]. В качестве альтернативных моделей рассматривались полиномы со степенями от 1 до 4.

Зависимость медианного времени восстановления всех деревьев составляющих для исходного предложения с данным числом словоформ представлена на рисунке 4.2 в логарифмической шкале

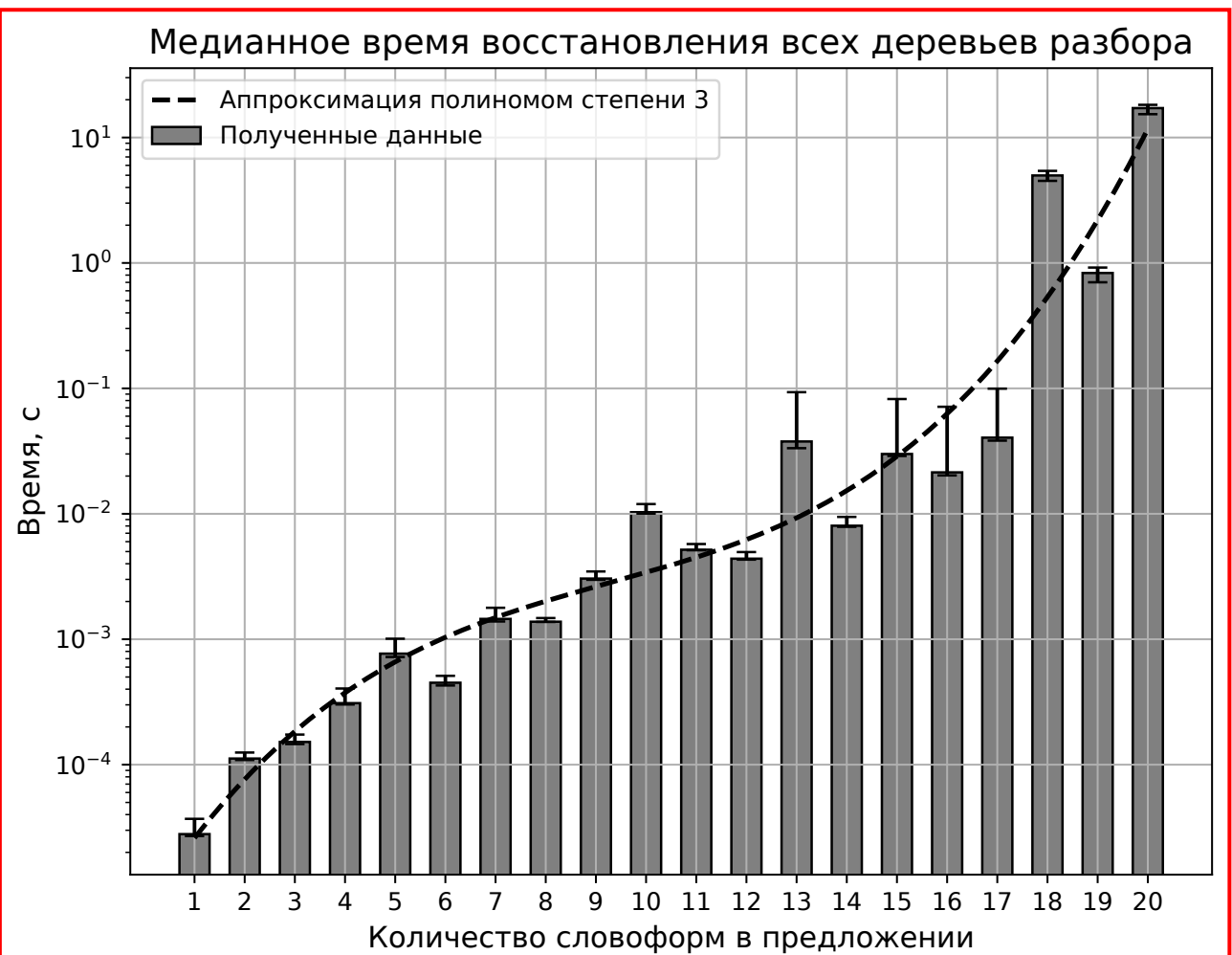


Рисунок 4.2 — Зависимость медианного времени выполнения алгоритма восстановления всех деревьев составляющих от числа словформ во входном предложении

Полученные данные были аналогично аппроксимированы полиномом третьей степени:

$$T_{tree}(n) \approx 4.52 \cdot 10^{-3} n^3 - 1.26 \cdot 10^{-1} n^2 + 1.42n - 11.86 \quad (4.6)$$

В таблице 4.1 приведена также зависимость числа полученных деревьев составляющих от количества словформ во входном предложении.

Таблица 4.1 — Зависимость числа восстановленных деревьев от числа словоформ в исходном предложении

Число словоформ в исходном предложении	Число восстановленных деревьев составляющих
1	1
2	3
3	2
4	4
5	16
6	8
7	96
8	44
9	268
10	1436
11	362
12	480
13	11032
14	1600
15	8748
16	3362
17	13374
18	937755
19	157544
20	2776928

На основе 500 выбранных предложений корпуса [41] с количеством словоформ, не превосходящим 15, были получены средние значения:

- $Precision = 0.68$;
- $Recall = 0.71$;
- $F1 = 0.69$.

Выводы

По результатам проведенного исследования наиболее продолжительным этапом в разработанном методе является восстановление всех возможных деревьев зависимостей, демонстрирующее экспоненциальный рост временных затрат. В общем случае с ростом числа словоформ наблюдается рост числа восстанавливаемых деревьев составляющих. Отсутствие монотонности в данной зависимости объясняется различиями в синтаксической структуре используемых в ходе исследования предложений. Аппроксимация времени выполнения этапа построения таблицы составляющих полиномом третьей степени демонстрирует соответствие асимптотике исходного алгоритма Кока–Янгера–Касами $O(n^3)$.

Разработанный метод верно определяет 71% синтаксических групп эталонного предложения, при этом 68% выделяемых методом синтаксических групп являются истинными. В работе [1] деревья составляющих строятся на основе готовых деревьев зависимостей, получаемых с помощью синтаксических анализаторов, значения точности полноты и F1-меры для каждого из которых приведены в таблице 4.2.



Вычисления в данной работе производились на основе 100 предложений из НКРЯ [5], по аналогичной используемой в рамках проводимого исследования методике.

Таблица 4.2 — Точность, полнота и F1-мера для различных синтаксических анализаторов в работе [1]

Синтаксический анализатор	Точность	Полнота	F1-мера
Stanza [43]	0.83	0.83	0.83
SpaCy [44]	0.70	0.69	0.70
Natasha [45]	0.68	0.49	0.57

Таким образом, разработанный метод уступает методу описанному в работе [1] с использованием синтаксического анализатора Stanza, но находится на одном уровне с тем же методом с использованием SpaCy и превосходит этот метод с использованием Natasha.



ЗАКЛЮЧЕНИЕ

Поставленная цель была достигнута: разработан метод построения синтаксических деревьев ограниченного естественного русского языка на основе наиболее употребимых грамматик.

В ходе выполнения выпускной квалификационной работы были выполнены следующие задачи:

- проанализирована предметная область автоматизированного построения синтаксических деревьев ограниченного естественного русского языка: выделены и описаны основные формализмы; для реализации выбраны системы составляющих;
- проведен сравнительный анализ существующих методов построения синтаксических деревьев: для использования в разрабатываемом методе выбран алгоритм Кока–Янгера–Касами;
- сформулированы требования к разрабатываемому методу;
- формализована постановка задачи в виде IDEF0-диаграммы;
- сформулированы ограничения предметной области;
- разработан метод построения синтаксических деревьев ограниченного естественного русского языка;
- обоснован выбор программных средств реализации и разработано программное обеспечение реализующее описанный метод;
- определена зависимость времени построения синтаксических деревьев от количества слов-форм в предложении: наиболее продолжительным этапом является восстановление деревьев составляющих, демонстрирующее экспоненциальный рост временных затрат;
- оценена точность и полнота разработанного метода: точность составила 0.68, полнота — 0.71, что соответствует уровню существующих решений для построения деревьев составляющих на основе готовых деревьев зависимостей.

Дальнейшее развитие может быть достигнуто за счет:

- более высокой «детализации» используемой грамматики, выделения подъединиц текущих синтаксических единиц, ввода новых правил грамматики;
- использования более подробного морфологического анализатора, охватывающего все существующие в языке части речи и их морфологические характеристики;
- внедрения возможности автоматического разрешения неоднозначности среди всех возвращаемых методом деревьев составляющих;
- оптимизации временных затрат этапа восстановления деревьев составляющих.



СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Полетаев А. Ю., Парамонов И. В., Бойчук Е. И. Алгоритм построения дерева синтаксических единиц русскоязычного предложения по дереву синтаксических связей // Информатика и автоматизация. 2023. Т. 22, № 6. С. 1323–1353.
2. Закирова К., Новикова И., Блюдова В [и др.]. Автоматический синтаксический анализ. НИУ ВШЭ в Нижнем Новгороде: Новости образовательной программы “Фундаментальная и прикладная лингвистика”. 2023. [Электронный ресурс]. Дата обращения: 15.01.2026. URL: <https://nnov.hse.ru/ba/ling/news/808162873.html>.
3. Зобнин А. И., Сахарова А. В. Универсальная система синтаксической разметки текстов ObjectATE // Компьютерная лингвистика и интеллектуальные технологии / под ред. А. Е. Кибрик, др. М.: РГГУ, 2009. С. 120–126.
4. СинТагРус сегодня / Е. С. Иншакова, Л. Л. Иомдин, Л. Г. Митюшин [и др.] // Труды Института русского языка им. В.В. Виноградова. Вып. 21. Национальный корпус русского языка: исследования и разработки. М., 2019. С. 14–40.
5. Национальный корпус русского языка. <https://ruscorpora.ru/>. 2026. [Электронный ресурс]. Дата обращения: 24.01.2026.
6. Гладкий А. В. Синтаксические структуры естественного языка в автоматизированных системах общения. Проблемы искусственного интеллекта. М.: Наука. Главная редакция физико-математической литературы, 1985. с. 144.
7. Гращенков П. В. RuConst: синтаксический корпус русского языка с разметкой по непосредственным составляющим // Вестник Московского университета. Серия 9. Филология. 2024. № 3. С. 94–112.
8. Демидов Д. В., Евченко И. В. Создание корпуса русских текстов с разметкой систем синтаксических групп // International Journal of Open Information Technologies. 2024. Т. 12, № 5. С. 33–38.
9. Естественно-языковые интерфейсы интеллектуальных вопросно-ответных систем / В. А. Житко, В. Н. Вяльцев, Ю. С. Гецевич [и др.]. 2011.
10. Захаров В. П., Богданова С. Ю. Корпусная лингвистика: учебник для студентов гуманитарных вузов. Иркутск: ИГЛУ, 2011. 161 с., ISBN 978-5-88267-316-0.
11. Leech Geoffrey. Introducing corpus annotation. 1997.
12. Копотев М. В. Введение в корпусную лингвистику. Прага: Animedia Company, 2014. 231 с., ISBN: 978-80-7499-067-0.
13. Treebanks: Building and Using Parsed Corpora / под ред. Anne Abeillé. Dordrecht: Springer, 2003. Т. 20 из Text, Speech and Language Technology.
14. Автоматическая обработка текстов на естественном языке и анализ данных / Е. И. Большакова,

15. Nivre Joakim. An Efficient Algorithm for Projective Dependency Parsing // Proceedings of the 7th Conference on Natural Language Learning at HLT-NAACL 2003. Edmonton, Canada: Association for Computational Linguistics, 2003. С. 1–8.
16. MaltParser: A Language-Independent System for Data-Driven Dependency Parsing / Joakim Nivre, Johan Hall, Jens Nilsson [и др.] // Natural Language Engineering. 2007. Т. 13, № 2. С. 95–135.
17. Киселёв М. В., Федосеева Д. В. Синтаксический парсер русского языка LPaRus компании Megaruter Intelligence // Компьютерная лингвистика и интеллектуальные технологии: По материалам ежегодной Международной конференции «Диалог». Москва: РГГУ, 2017.
18. Тестелец Я. Г. Введение в общий синтаксис. М.: Российский государственный гуманитарный университет, 2001.
19. Jurafsky Daniel, Martin James H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. Third draft изд. 2026. Draft of January 6, 2026.
20. Гладкий А. В. К процедуре построения систем синтаксических групп // Московский лингвистический журнал. М., 1998. Т. 4. С. 32–45.
21. Коротаев Н. А. Синтаксические группы А. В. Гладкого: анализ конструкций с сочинением // Вестник Российского государственного гуманитарного университета. Серия: Литературоведение. Языкознание. Культурология. М., 2013. № 8. С. 17–36.
22. Makarov V. Cocke–Younger–Kasami–Schwartz–Zippel algorithm and relatives. 2022.
23. Earley Jay. An Efficient Context-Free Parsing Algorithm // Communications of the ACM. 1970. Т. 13, № 2. С. 94–104.
24. Younger Daniel H. Recognition and parsing of context-free languages in time n^3 // Information and Control. 1967. Т. 10, № 2. С. 189–208.
25. Национальный корпус русского языка. Синтаксическая разметка в корпусе СинТагРус. 2025. [Электронный ресурс]. Дата обращения: 20.03.2026. URL: https://ruscorpora.ru/file/annotation_syntagrus/.
26. Аракин В. Д. Сравнительная типология английского и русского языков : учебное пособие. 4-е изд. изд. Москва: ФИЗМАТЛИТ, 2010. с. 232.
27. Гак В. Г. Сравнительная типология французского и русского языков : учеб. для пед. ин-тов по спец. N 2103 «Иностр. яз.». 2-е изд. изд. Москва: Просвещение, 1983. с. 287.
28. Штайн Т. Кормен Ч. Лейзерсон Р. Ривест К. Алгоритмы: построение и анализ. Москва: ООО "И. Д. Вильямс 2013. 1328 с., ISBN 978-5-8459-1794-2.
29. Bray T. The JavaScript Object Notation (JSON) Data Interchange Format: Tech. Rep.: RFC 8259: Internet Engineering Task Force (IETF), 2017. [Электронный ресурс]. Дата обращения: 17.04.2026. URL:

30. Segalovich Ilya. A fast morphological algorithm with unknown word guessing induced by a dictionary for a web search engine. 2003. [Электронный ресурс]. Дата обращения: 30.11.2025. URL: <https://cloudcdn-m9-12.cdn.yandex.net/download.yandex.ru/company/iseg-las-vegas.pdf?lid=7>.
31. Kovalenko A. Stemka — Russian and Ukrainian morphological guesser. 2002. [Электронный ресурс]. Дата обращения: 10.12.2025. URL: <http://www.keva.ru/stemka/stemka.html>.
32. Сапин А. С. Методы и средства морфологической сегментации для систем автоматической обработки текстов. Диссертация на соискание учёной степени кандидата физико-математических наук: Национальный исследовательский университет «Высшая школа экономики». Москва, 2023.
33. Pymorphy Development Team. pymorphy3: Documentation. [Электронный ресурс]. Дата обращения: 20.12.2025. URL: <https://github.com/no-plagiarism/pymorphy3/tree/master/docs>.
34. Python Software Foundation. The Python Tutorial. Python Software Foundation. 2025. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://docs.python.org/3/tutorial/>.
35. Ramirez Sebastian. FastAPI GitHub Repository. 2025. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://github.com/fastapi/fastapi/tree/master/docs/ru/docs>.
36. TypeScript Contributors. TypeScript Documentation. 2026. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://www.typescriptlang.org/docs/>.
37. Meta Platforms, Inc. React Reference Documentation. 2026. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://react.dev/reference/react>.
38. Intel Corporation. Intel(R) Core(TM) i5-10210U Processor (6M Cache, up to 4.20 GHz) Specifications. 2019. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://www.intel.com/content/www/us/en/products/sku/195436/intel-core-i510210u-processor-6m-cache-up-to-4-20-ghz/specifications.html>.
39. Canonical Ltd. Ubuntu Desktop 24.04 Documentation. 2024. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://documentation.ubuntu.com/desktop/en/24.04/>.
40. Сложность российских законов. Опыт синтаксического анализа / А. В. Кнутов, С. М. Плаксин, Р. Х. Синятуллин [и др.]. Москва: Издательский дом Высшей школы экономики, 2020. с. 311.
41. Гращенков Павел Валерьевич. Ru_Const: Синтаксический корпус РЯ на составляющих и поисковый инструмент к нему. 2023. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: https://github.com/grapaul/Ru_Const.
42. Burnham Kenneth P., Anderson David R. Model Selection and Multimodel Inference: A Practical Information-Theoretic Approach. 2 изд. New York: Springer-Verlag, 2002.
43. Qi Peng, Zhang Yuhao, Zhang Yuhui [и др.]. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. 2020.



44. spaCy: Industrial-strength Natural Language Processing in Python / Matthew Honnibal, Ines Montani, Sofie Van Landeghem [и др.]. 2020. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://spacy.io/usage>.
45. Koltun Alexander [и др.]. Natasha: Solves basic Russian NLP tasks, API for lower level Natasha projects. 2025. [Электронный ресурс]. Дата обращения: 20.05.2026. URL: <https://github.com/natasha/natasha>.



ПРИЛОЖЕНИЕ А

Реализуемые правила грамматики ограниченного естественного русского языка.

Таблица 4.3 — Реализуемые правила грамматики ограниченного естественного русского языка

Формальная запись правила	Описание
$IP \rightarrow NP VP$ $IP \rightarrow VP NP$	Группа существительного соединяется с глагольной группой образуя предикативную основу предложения
$IP \rightarrow NP IP$	Именная группа предшествует предикативной основе предложения, дополняя ее
$IP \rightarrow NP ConjVP$	Именная группа соединяется с глагольной группой посредством запятой или союза, образуя предикативную основу предложения
$IP \rightarrow NP AP$ $IP \rightarrow AP NP$	Именная группа соединяется с группой прилагательного образуя предикативную основу без глагола в роли сказуемого
$IP \rightarrow NP NP$	Именная группа соединяется с другой именной группой образуя предикативную основу без глагола в роли сказуемого
$IP \rightarrow PP NP$ $IP \rightarrow NP PP$	Именная группа соединяется с предложной группой образуя предикативную основу без глагола в роли сказуемого
$IP \rightarrow C IP$	Предикативная основа предложения дополняется служебным словом слева
$IP \rightarrow C ConjIP$	Предикативная основа предложения дополняется служебным словом слева и соединяется с ним посредством союза или запятой
$IP \rightarrow AdvP IP$	Предикативная основа предложения дополняется группой наречия слева
$IP \rightarrow IP ConjIP$	Предикативная основа предложения соединяется с другой посредством запятой, двоеточия, точки с запятой или союза
$IP \rightarrow VP$ $IP \rightarrow AdvP$ $IP \rightarrow AP$ $IP \rightarrow NP$ $IP \rightarrow PP$	Предикативная основа предложения выражена единственной синтаксической группой
$VP \rightarrow V$	Глагольная группа состоит из одного глагола
$VP \rightarrow V NP$ $VP \rightarrow NP V$	Глагол соединяется с именной группой, образуя глагольную группу
$VP \rightarrow V PP$ $VP \rightarrow PP V$	Глагол соединяется с предложной группой, образуя глагольную группу
$VP \rightarrow VP PP$ $VP \rightarrow PP VP$	Глагольная группа дополняется предложной группой



Продолжение таблицы 4.3 — Реализуемые правила грамматики ограниченного естественного русского языка

Формальная запись правила	Описание
$VP \rightarrow VP AdvP$ $VP \rightarrow AdvP VP$	Глагольная группа дополняется группой наречия
$VP \rightarrow V CP$ $VP \rightarrow CP V$ $VP \rightarrow V ConjCP$	Глагол соединяется с частью предложения, вводимой союзом, либо с присоединенной частью после запятой
$VP \rightarrow VP ConjVP$ $VP \rightarrow VP VP$	Две глагольные группы соединяются союзом, запятой или бессоюзной связью
$VP \rightarrow V NP PP$ $VP \rightarrow V NP NP$	После глагола последовательно дополняется именной и предложной группой или двумя именными группами
$NP \rightarrow Pron$ $NP \rightarrow N$	Именная группа состоит из одного местоимения или одного существительного
$NP \rightarrow AP N$ $NP \rightarrow N AP$	Существительное соединяется с описательной группой, образуя именную группу
$NP \rightarrow AP NP$	Именная группа дополняется описательной группой слева
$NP \rightarrow AP ConjN$	Существительное соединяется с описательной группой слева через запятую или союз, образуя именную группу
$NP \rightarrow Q N$ $NP \rightarrow N Q$	Числительное соединяется с существительным, образуя именную группу
$NP \rightarrow N N$	Существительное соединяется с другим существительным образуя именную группу
$NP \rightarrow N NP$	Именная группа дополняется существительным слева
$NP \rightarrow N PP$ $NP \rightarrow N CP$ $NP \rightarrow N ConjAP$	Существительного соединяется справа с предложной группой, частью предложения, или описательной группой через союз, образуя именную группу
$NP \rightarrow NP ConjNP$ $NP \rightarrow NP NP$ $NP \rightarrow NP ConjCP$	Именная группа дополняется другой именной группой, либо частью предложения посредством союза или запятой
$PP \rightarrow P NP$ $PP \rightarrow P Q$	Предлог соединяется справа с именной группой или числительным, образуя предложную группу
$PP \rightarrow PP ConjPP$	Предложная группа другой предложной группой посредством союза или запятой
$AP \rightarrow A$	Группа прилагательного состоит из одного слова со значением признака
$AP \rightarrow AdvP A$ $AP \rightarrow A AdvP$	Прилагательное соединяется с группой наречия, образуя группу прилагательного
$AP \rightarrow A PP$ $AP \rightarrow PP A$	Прилагательное соединяется с предложной группой, образуя группу прилагательного

Продолжение таблицы 4.3 — Реализуемые правила грамматики ограниченного естественного русского языка

Формальная запись правила	Описание
$AP \rightarrow A N$	Прилагательное соединяется с существительным образуя группу прилагательного
$AP \rightarrow AP NP$	Группа прилагательного дополняется группой существительного справа
$AP \rightarrow AP ConjPP$	Группа прилагательного дополняется предложной группой справа посредством союза
$AP \rightarrow AP ConjAP$ $AP \rightarrow AP AP$	Группа прилагательного дополняется другой группой прилагательного посредством союза или запятой
$AdvP \rightarrow Adv$	Группа наречия состоит из одного наречия
$AdvP \rightarrow AdvP AdvP$ $AdvP \rightarrow AdvP NP$ $AdvP \rightarrow AdvP ConjAdvP$	Группа наречия дополняется другой группой наречия посредством союза или запятой или именной группой
$CP \rightarrow C IP$	Служебное слово соединяется с предикативной основой предложения, образуя зависимую часть предложения
$ConjIP \rightarrow Comma IP$ $ConjIP \rightarrow Semicolon IP$ $ConjIP \rightarrow Colon IP$ $ConjIP \rightarrow C IP$ $ConjIP \rightarrow CommaC IP$	Предикативная основа дополняется слева знаком препинания или служебным словом
$CommaC \rightarrow Comma C$	Запятая соединяется с союзом образуя единую синтаксическую единицу
$ConjVP \rightarrow C VP$ $ConjVP \rightarrow Comma VP$	Глагольная группа соединяется слева с союзом или запятой, образуя присоединительную глагольную группу
$ConjNP \rightarrow C NP$ $ConjNP \rightarrow Comma NP$	Именная группа соединяется слева с союзом или запятой, образуя присоединительную именную группу
$ConjPP \rightarrow C PP$ $ConjPP \rightarrow Comma PP$	Предложная группа соединяется слева с союзом или запятой, образуя присоединительную предложную группу
$ConjAP \rightarrow C AP$ $ConjAP \rightarrow Comma AP$	Группа прилагательного соединяется слева с союзом или запятой, образуя присоединительную группу прилагательного
$ConjAdvP \rightarrow C AdvP$ $ConjAdvP \rightarrow Comma AdvP$	Группа наречия соединяется слева с союзом или запятой, образуя присоединительную группу наречия
$ConjCP \rightarrow Comma CP$	Часть предложения соединяется слева с запятой, образуя присоединительную часть предложения

ПРИЛОЖЕНИЕ Б

Правила согласования применяемые в ограниченном русском языке.

Таблица 4.4 — Правила согласования ограниченного естественного русского языка

Формальная запись правила согласования	Описание
$IP : NP \leftrightarrow VP$ $IP : VP \leftrightarrow NP$ $IP : NP \leftrightarrow ConjVP$	Именная и глагольная группы согласуются в числе. Дополнительно в прошедшем времени согласование в роде, в настоящем времени — в лице. Если глагольная группа предшествует именной и выражена неопределенной формой, именная группа рассматривается как дополнение
$IP : NP \leftrightarrow AP$ $IP : AP \leftrightarrow NP$	В предложении без глагола в роли сказуемого именная группа и группа прилагательного согласуются в роде, числе и падеже
$IP : NP \leftrightarrow NP$	В предложении без глагола в роли сказуемого две именные группы согласуются в числе и роде
$IP : NP \leftrightarrow PP$ $IP : PP \leftrightarrow NP$	В сочетании именной и предложной группы именная группа принимает форму именительного падежа
$IP : NP_{nomn} \leftrightarrow IP$	Именная группа в именительном падеже перед предикативной основой согласуется с ней в числе; в прошедшем времени — в роде; в настоящем времени — в лице
$NP : AP \leftrightarrow N$ $NP : AP \leftrightarrow NP$ $NP : AP \leftrightarrow ConjN$ $NP : N \leftrightarrow AP$	Группа прилагательного согласуется с именной группой или присоединительной именной группой, или существительным в роде, числе и падеже
$NP : N \rightarrow N_{gent}$ $NP : N \rightarrow NP_{gent}$	Существительное или именная группа после существительного принимают форму родительного падежа
$VP : V \rightarrow NP$	Именная группа после глагола не принимает форму именительного падежа. После переходных глаголов именная группа принимает форму только винительного падежа
$PP : P \rightarrow NP$	Падеж именной группы после предлога определяется видом предлога на основе словаря
$NP : NP \leftrightarrow NP$ $NP : NP \leftrightarrow ConjNP$	Две именные группы согласуются в падеже
$VP : VP \leftrightarrow VP$ $VP : VP \leftrightarrow ConjVP$	Две глагольные группы согласуются во времени, числе и лице
$AP : AP \leftrightarrow AP$ $AP : AP \leftrightarrow ConjAP$	Две группы прилагательных согласуются в роде, числе и падеже
$IP : VP \not\leftrightarrow infn$	Неопределенная форма глагола не может быть использована как предикативная основа



Продолжение таблицы 4.4 — Правила согласования ограниченного естественного русского языка

Формальная запись правила согласования	Описание
$NP : Q \leftrightarrow N$ $NP : N \leftrightarrow Q$	Числительное и существительное согласуются в роде числа
$NP : Q \leftrightarrow AP \leftrightarrow N$	В именной группе с числительным группа прилагательного согласуется с ним в числе и падеже

ПРИЛОЖЕНИЕ В

Презентация к выпускной квалификационной работе на тему «Метод построения синтаксических деревьев для ограниченного естественного русского языка на основе наиболее употребимых грамматик» из 8 слайдов.

